Software Factory & Developer Experience

Version 1, 2026-07-30

Introduction

Welcome to this quick course on the ***Software Factory & Developer Experience***. This course is the `second` in a series of `six` courses about **Application Platform Delivery**:

- *Application Platform Delivery Foundations*
- *Software Factory & Developer Experience*(*This course*)
- *Software Factory and CI/CD*
- *Application Modernization*
- *Developer Hub and Platform Engineering*
- *Trusted Software Supply Chain (TSSC)*

What's in this course?

By the end of this module, you will be able to deploy, configure, and use Red Hat OpenShift Dev Spaces to develop, build, and deploy applications through integrated GitLab and CI/CD workflows.

Duration: 2.5 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- .. (SME)
- .. (SME)
- Yogita Soni (Content Architect)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

Based on the section index, here are concise learning objectives:

Learning Objectives

By the end of this module, you will be able to:

- Describe the purpose and architecture of Red Hat OpenShift Dev Spaces.
- Explain the key concepts and core components of Dev Spaces.
- Understand how Dev Spaces fits into a Software Factory and supports cloud-native application development.

- Identify the supporting services required to enable an integrated developer platform.
- Deploy and configure the Red Hat OpenShift Dev Spaces Operator on an OpenShift cluster.
- Integrate Dev Spaces with GitLab to enable source code management and developer workflows.
- Create and use Dev Spaces workspaces for cloud-native application development.
- Understand how Continuous Integration (CI) pipelines are triggered and executed from developer workspaces.
- Deploy applications to OpenShift using automated delivery workflows.
- Explain the role of promotion pipelines in advancing applications across development, testing, and production environments.

Prerequisites

This course assumes that you have the following prior experience:

1. Red Hat OpenShift Container Platform (OCP) Technical Overview or Kubernetes Basics

Lab Environment

Click the RHDP link above, log in to RHDP, and click `Order` to request a new Service Enablement: App Platform Software Factory. On the order page for the catalog item, do the following:

- **Activity**: Select `Practice/Enablement`
- **Purpose**: Select `Learning about the Product`
- **Salesforce ID**: If you have an opportunity ID, enter it into this field, otherwise select `Project`, and then enter "Learning about RHCL" in the text field. You will see a warning message that a Salesforce ID is required, but you can safely ignore it.
- **Features**: Select `Enable Let's Encrypt`
- **Region**: Select the AWS region closest to your location
- **User count**: `1`
- **Control Plane Count**: `1`
- **OpenShift Version**: `4.16`
- **Control Plane Instance Type**: `m6a.4xlarge`
- Select the checkbox at the bottom of the page to confirm that lab costs will be charged to your cost center, and click `Order`.

The lab environment will take an hour to provision. You can check the status and details of the OpenShift cluster from the `Services` page of RHDP. The classroom is available for 48 hours initially. You can extend the duration of classroom availability by clicking `Help > Get Technical Help` in RHDP, and then requesting an extension by opening a ServiceNow ticket with the RHDP team.

RHDP Portal Links

- RedHat Associates
- RedHat Partners

Software Factory Overview

Introduction

In the previous module, you learned how an **Application Platform** provides the capabilities required to build, deploy, secure, and operate modern applications.

In this lesson, you'll discover how organizations use those capabilities to create a **Software Factory**—a standardized software delivery process that enables developers to build and deliver applications quickly, securely, and consistently.

As you progress through this lesson, think about how a new developer joins your organization today. How many manual steps are required before they can deploy their first application?

The Challenge

Every development team wants to deliver software quickly.

Unfortunately, many organizations evolve over time, resulting in different teams using different tools, different processes, and different engineering practices.

A new developer joining the organization often needs to answer questions such as:

- Which Git repository should I use?
- Which build pipeline does this team use?
- How do I deploy my application?
- Where is the documentation?
- Which security requirements must I follow?
- Who do I contact if something goes wrong?

Instead of writing code, developers spend valuable time learning processes and integrating tools.

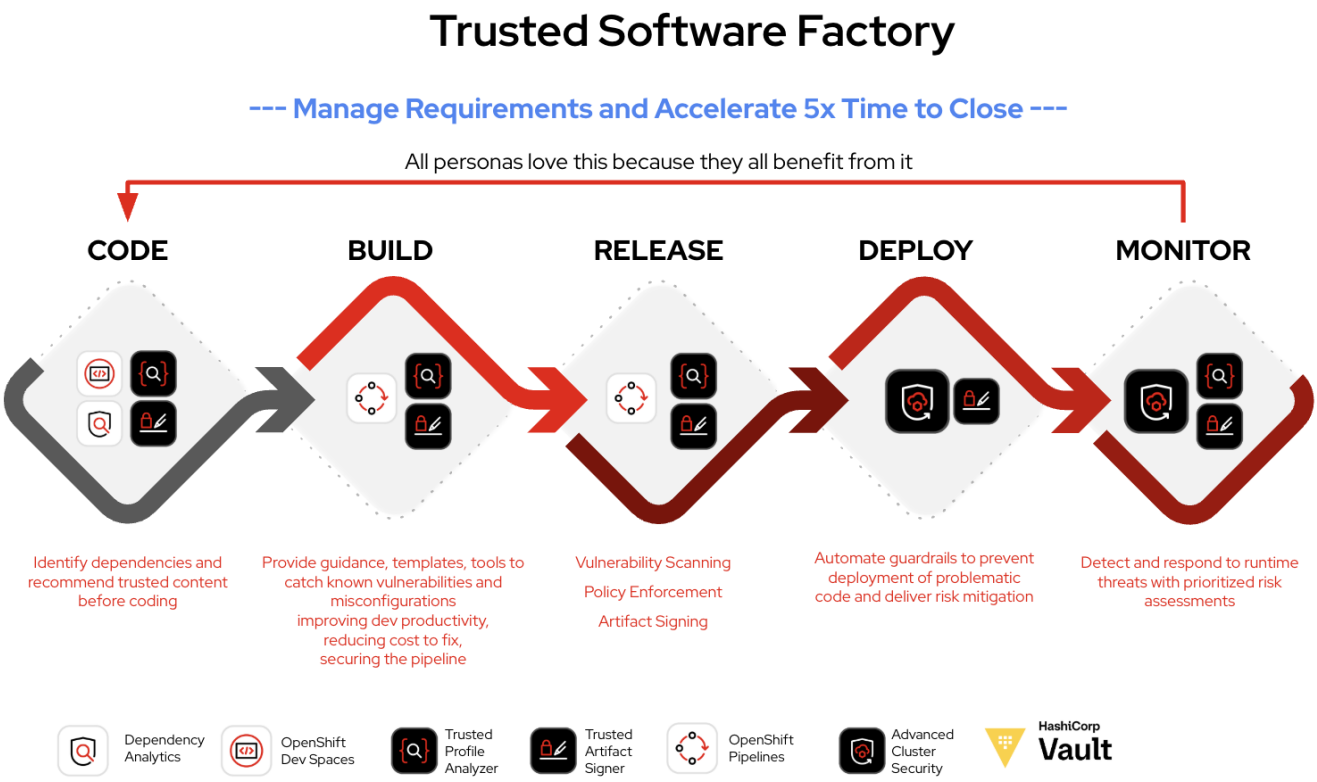
When every team builds its own software delivery process, developer onboarding becomes slower, operational complexity increases, and engineering practices become inconsistent.

What is a Software Factory?

A **Software Factory** is a standardized approach for building, testing, securing, deploying, and operating software.

Rather than every development team assembling its own toolchain, Platform Engineers provide a curated platform that offers reusable services, automation, and organizational guardrails.

Developers simply consume these capabilities through a consistent self-service experience.



Software Delivery Lifecycle

As application code moves from development to production, the Software Factory performs many tasks automatically.

Stage	Platform Activities
Code	Developers use standardized workspaces, templates, and source control.
Build	Applications are built, dependencies are validated, and security checks run automatically.
Release	Artifacts are signed, verified, and promoted according to organizational policies.
Deploy	Applications are deployed using automated pipelines and GitOps workflows.
Monitor	The platform continuously monitors application health, logs, and runtime behavior.

Business Value

By standardizing software delivery, organizations gain:

- Faster developer onboarding
- Consistent engineering practices
- Self-service development workflows
- Built-in security and compliance

- Automated software delivery
- Reduced operational complexity

A **Software Factory** is not a single product.

It is a standardized software delivery approach built on an Application Platform that combines automation, reusable services, and organizational best practices into a consistent developer experience.

Reflection

▼ *Think about your own organization (Click to expand)*

- Does every development team follow the same software delivery process?
- Which activities are already automated?
- Which steps still require manual effort?
- How could a standardized Software Factory improve developer productivity?

Check Your Understanding

▼ *Quiz Time! (Click to expand)*

Which statement best describes a Software Factory?

1. Every development team builds and maintains its own toolchain.
2. A standardized platform that automates how software is built, secured, deployed, and operated.
3. A collection of independent tools managed separately by each team.

Answer

B — A Software Factory provides standardized workflows, automation, reusable services, and organizational guardrails for software delivery.

Key Takeaway

A **Software Factory** standardizes software delivery by providing reusable workflows, automation, and built-in engineering practices, allowing developers to focus on building applications instead of integrating tools.

A Day in the Software Factory

Now that you've learned what a Software Factory is, let's see how it helps a developer during everyday software development.

In this lesson, you'll follow Emma as she delivers her first application using her organization's Software Factory.

As you read through Emma's journey, think about how similar steps are performed in your own organization.

Meet Emma

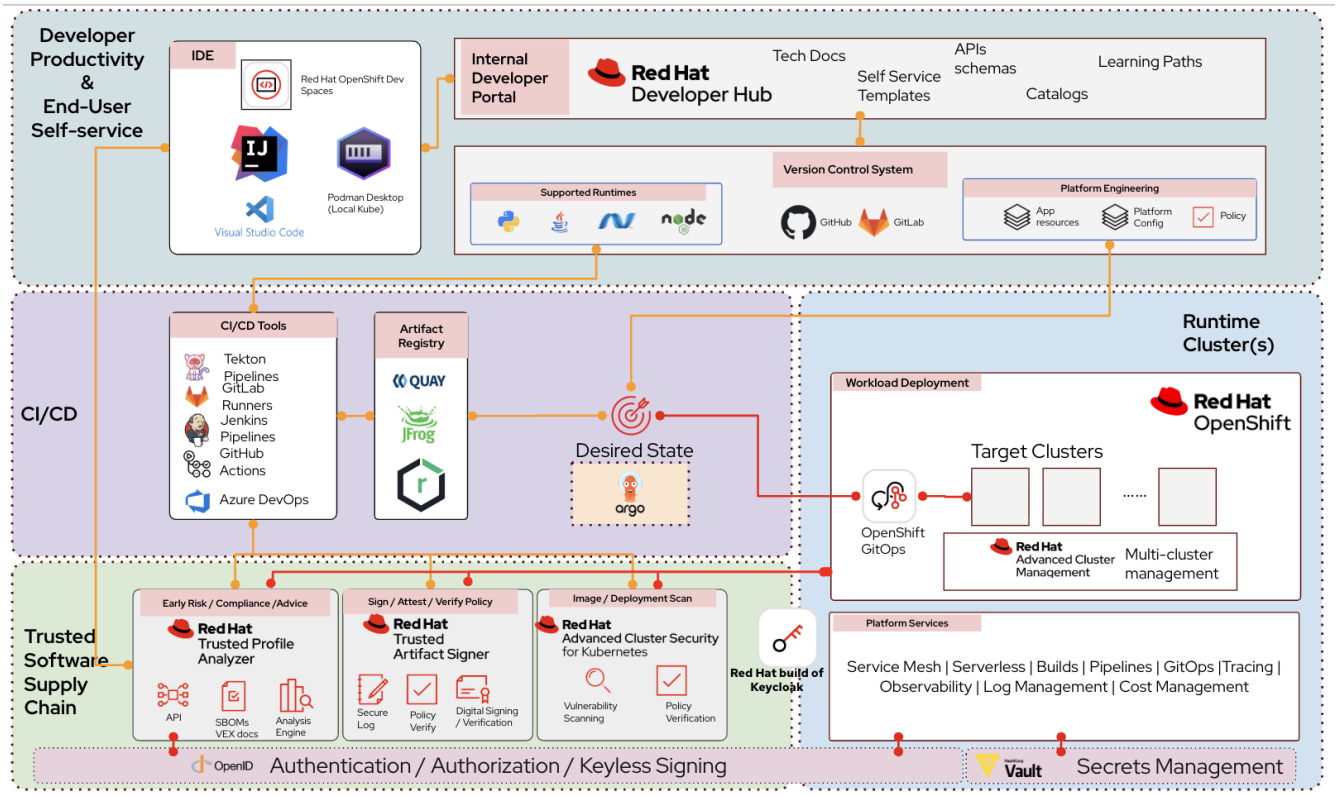
Emma has just joined the engineering team.

She has been assigned her first feature and wants to start contributing immediately.

Instead of requesting infrastructure, configuring development environments, creating CI/CD pipelines, and learning deployment processes, Emma uses the organization's Software Factory.

The platform provides everything she needs through a standardized self-service experience.

Emma's Journey



▼ *Emma's workflow (Click to expand)*

Emma's first day looks like this:

1. Sign in to the organization's **Developer Hub**.
2. Choose a **Golden Path** for the application she wants to build.
3. Launch a ready-to-use development workspace using **Dev Spaces**.
4. Write code and commit the changes to Git.
5. The platform automatically builds and validates the application.
6. Security and compliance checks run automatically.
7. GitOps deploys the application to OpenShift.
8. The platform continuously monitors the running application.

Behind Every Step

Emma interacts with only a few simple interfaces, but many platform capabilities work together behind the scenes.

Emma's Activity	Platform Capability
Find project templates	Developer Hub
Start coding	Dev Spaces
Store source code	GitHub or GitLab
Build and test applications	OpenShift Pipelines
Perform security validation	Trusted Software Supply Chain
Store trusted artifacts	Quay
Deploy applications	GitOps
Monitor running applications	Platform Services

Why This Matters

Without a Software Factory, Emma would need to:

- Learn multiple development processes.
- Configure development tools.
- Build deployment pipelines.
- Understand infrastructure.
- Integrate security tooling.

With a Software Factory, these capabilities are already provided by the platform.

Emma focuses on delivering business features while the platform handles the underlying automation, security, and operational tasks.

A Software Factory doesn't replace developers—it removes the repetitive platform work that slows them down, allowing developers to focus on writing application

> code.

Reflection

▼ *Think about your own organization (Click to expand)*

When a new developer joins your team:

- How long does onboarding take?
- Is there a standardized workflow?
- Which activities are manual?
- Which activities could be automated through a Software Factory?

Check Your Understanding

▼ *Quiz Time! (Click to expand)*

Match Emma's activity with the platform capability that supports it.

Activity	**Capability**
Find project templates	?
Start coding immediately	?
Automatically build applications	?
Deploy consistently	?
Store trusted container images	?

Answers

- Find project templates → Developer Hub
- Start coding immediately → Dev Spaces
- Automatically build applications → OpenShift Pipelines
- Deploy consistently → GitOps
- Store trusted container images → Quay

Version Control

Introduction

In this lesson, you'll learn why version control is a foundational capability of a modern Application Platform and how it enables collaborative software development, CI/CD, and GitOps.

If you're already comfortable using Git and branching strategies, feel free to skim this lesson and continue to the hands-on lab. The concepts introduced here provide useful context for the Software Factory you'll build throughout this course.

Connecting the Dots

Once developers write code, they need a reliable way to share changes, review code, maintain version history, and trigger automated software delivery workflows.

This is where **Git** becomes an essential part of the Software Factory.

[GitHub Flow] |
*https://miro.medium.com/v2/resize:fit:720/format:webp/1*S4VJPf8Mrg3Gn5cR7JK62w.png*

Throughout this learning path, Git serves as the source of truth for your application code. Every code change committed to Git can automatically trigger builds, tests, deployments, and GitOps workflows.

Why Version Control Matters

Modern applications are rarely developed by a single developer.

Multiple developers often work on the same application simultaneously, making it essential to track changes, collaborate safely, and maintain a complete history of the application.

Version control systems such as **Git** make this possible by recording every change made to the source code.

Instead of storing only the latest version of an application, Git maintains a complete history that allows teams to:

- Collaborate without overwriting each other's work
- Track who made changes and when
- Review code before it is merged
- Restore previous versions when necessary
- Develop features independently
- Trigger automated software delivery pipelines

Key Takeaway

> Git is much more than a code repository. It provides the collaboration and version history that modern Application Platforms rely on to automate software delivery.

Understanding Branches

A **branch** represents an isolated line of development.

Rather than making changes directly to the main codebase, developers create a separate branch where they can safely implement new features, fix defects, or experiment with ideas.

Once the work has been reviewed and validated, the branch is merged back into the main branch.

[GitFlow] | *https://nvie.com/img/git-model@2x.png*

> Throughout the hands-on labs, you'll create feature branches, commit changes, and merge them into the main branch to trigger automated Software Factory workflows.

Common Branching Strategies

Different organizations adopt different branching models depending on their release process, team size, and deployment frequency.

Branching Strategy	Characteristics	Common Use Cases
Trunk-Based Development	Developers create short-lived branches and merge changes into the main branch frequently.	Continuous Integration and Continuous Delivery
GitHub Flow	Feature branches combined with Pull or Merge Requests before merging into the main branch.	Cloud-native applications and open source projects
GitFlow	Multiple long-lived branches for development, releases, and hotfixes.	Traditional enterprise applications with scheduled releases

Branching Strategy Used in This Course

Throughout this learning path, you'll use a simplified Trunk-Based Development workflow.

[Trunk-Based Development] | *https://trunkbaseddevelopment.com/trunk1b.png*

This approach keeps the focus on learning Application Platform concepts rather than Git administration.

The workflow you'll follow is:

1. Create a feature branch
2. Make code changes

3. Commit your changes
4. Push the branch to Git
5. Merge the branch into the main branch
6. Automatically trigger the Software Factory

You'll practice this workflow during the upcoming hands-on lab, where each Git commit becomes the starting point for automated build and deployment pipelines.

Key Takeaway

In a modern Software Factory, **Git is often the event that starts the entire software delivery process**. A single commit can automatically trigger builds, testing, security validation, deployments, and GitOps synchronization.

Reflection

▼ *Think about your own organization (Click to expand)*

- Which version control system does your organization use?
- Which branching strategy is followed by your team?
- Are code changes automatically validated through CI/CD?
- How frequently are changes merged into the main branch?

Check Your Understanding

▼ *Quiz Time! (Click to expand)*

- Why do developers create feature branches?
  1. To replace the main branch
  2. To isolate changes before merging them into the shared codebase
  3. To improve Kubernetes performance
  4. To avoid using version control

Answer

B — Feature branches allow developers to work independently without affecting the shared codebase until their changes are reviewed and merged.

- Which branching strategy is primarily used throughout this learning path?
  1. GitFlow
  2. GitHub Flow
  3. Trunk-Based Development
  4. No branching

Answer

C — This course follows a simplified Trunk-Based Development approach with short-lived feature branches and frequent integration.

- Which Application Platform capabilities commonly begin with a Git commit?

Capability	Starts from Git?
CI/CD Pipeline	Yes
GitOps Deployment	Yes
Security Scanning	Yes
Application Build	Yes
Monitoring Existing Applications	No

Lab: Environment Overview

Before starting the hands-on exercises, familiarize yourself with the lab environment and the services available throughout this learning path.

The core infrastructure has already been deployed and configured. During the remaining labs, you'll build upon this foundation by deploying additional platform capabilities and integrating them into a complete **Application Platform Software Factory**.

Lab Environment

The following platform services are pre-configured and ready to use throughout the lab.

Service	Purpose	Access
Red Hat Build of Keycloak (RHBK)	Provides centralized Single Sign-On (SSO) and identity management for the lab.	`KEYCLOAK_URL` *(Refer to the Lab Information page in RHDP.)*
GitLab CE	Hosts the application source code and Git repositories used during the lab.	`GITLAB_URL` *(Refer to the Lab Information page in RHDP.)*
Red Hat Quay	Enterprise container registry used to store and distribute container images.	`QUAY_URL` *(Refer to the Lab Information page in RHDP.)*
OpenShift GitOps	Provides GitOps-based continuous delivery and manages platform resources.	Pre-configured

Lab Credentials

Use the following user accounts throughout the lab.

Username	Password	Purpose
`platform-admin`	`openshift`	Platform administrator
`platform-developer`	`openshift`	Application developer
`platform-user`	`openshift`	Standard user

Unless otherwise instructed, authenticate to GitLab, Quay, and other integrated services using **Keycloak Single Sign-On (SSO)** with one of the lab accounts above.

OpenShift Clusters

The lab environment consists of two OpenShift clusters.

Cluster	Purpose
Primary Cluster	Development environment where you'll build, test, and deploy applications.
Secondary Cluster	Staging and Production environment used to demonstrate multi-cluster application promotion.

The required namespaces have already been created and are ready for use.

What You'll Deploy

Throughout this training, you'll deploy and configure the following application platform capabilities:

- PostgreSQL
- Red Hat AMQ Streams (Kafka)
- Red Hat OpenShift Pipelines (Tekton)
- Red Hat build of OpenTelemetry
- Red Hat OpenShift Dev Spaces
- Red Hat OpenShift GitOps (Argo CD)

By the end of the *learning path*, these components will be integrated to create a complete **Application Platform Software Factory**.

Core infrastructure components—including Red Hat Build of Keycloak, GitLab, Red Hat Quay, and HashiCorp Vault—have already been provisioned and do not require installation.

Lab1

In this lab, you'll access the Software Factory environment and prepare the application that will be used throughout the remaining exercises.

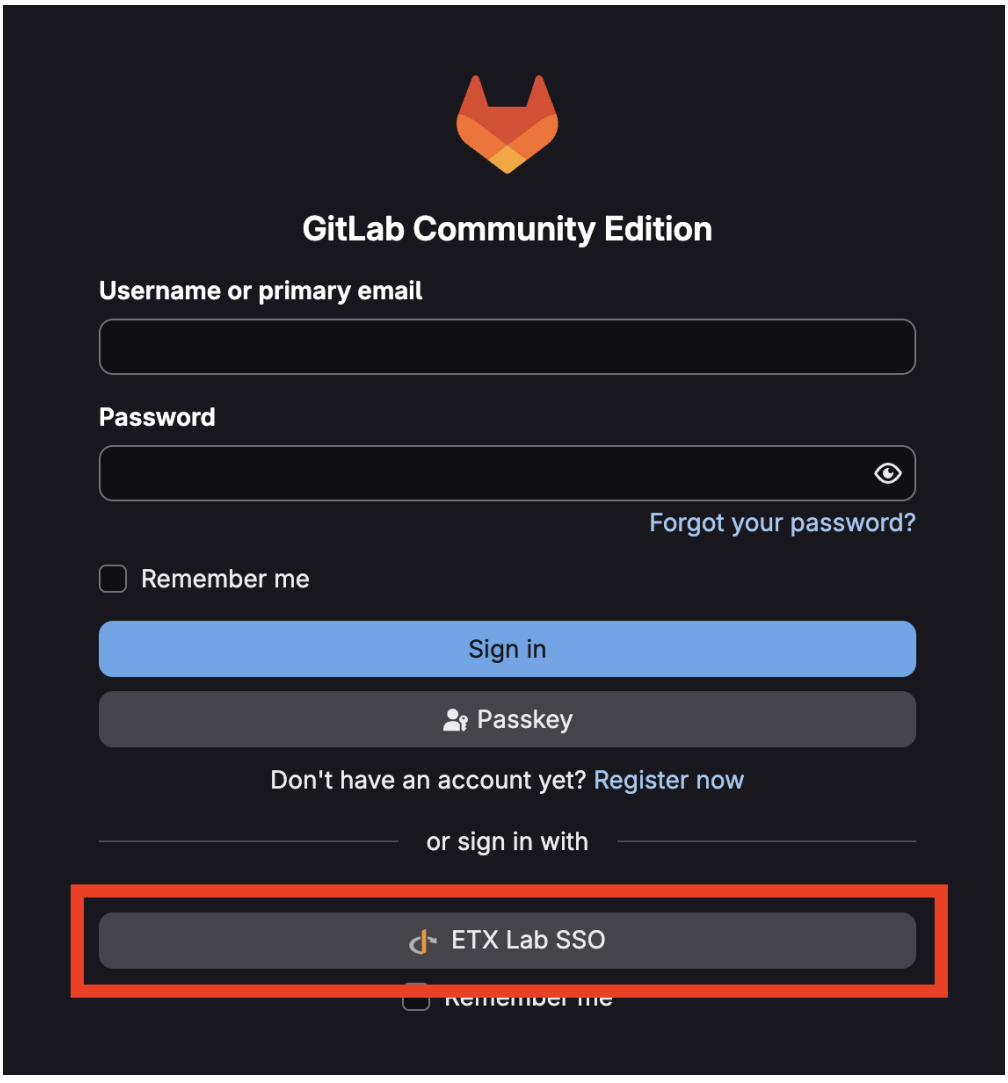
By the end of this lab, you will have:

- Authenticated to GitLab using enterprise Single Sign-On
- Cloned the workshop application repository
- Verified access to the OpenShift cluster

Sign in to GitLab

The workshop uses GitLab as the central source code repository. Authentication is handled through Keycloak using Single Sign-On (SSO).

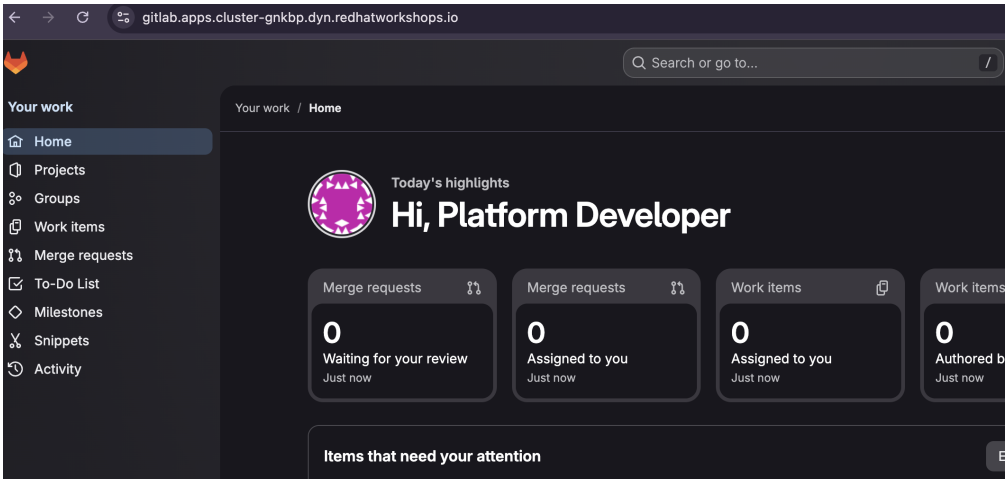
1. Navigate to the **GitLab URL** provided on your lab information page (for example, `https://gitlab.apps.cluster-gnkbp.dyn.redhatworkshops.io`).
2. Click btn:[ETX Lab SSO] as shown in the screenshot:



3. Sign in using the provided credentials:

   `platform-developer` / `openshift`

4. After successful authentication, you'll be redirected back to GitLab.



The first time you sign in, GitLab automatically creates a user profile linked to your Keycloak identity.

Clone the Workshop Repository

Throughout the training you'll work with the **ETX App Base** application.

Use the **OpenShift Web Terminal** to clone the repository.

<Add step around login into openshift console by taking it's access from lab environment page>

Open the Web Terminal by clicking the >_ icon in the upper-right corner of the OpenShift Console.

First, determine the GitLab URL.

```
GITLAB_URL="https://$(oc get route \
  -n gitlab \
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}')"

echo "${GITLAB_URL}"
```

Clone the application repository.

```
mkdir -p ~/workshop
cd ~/workshop

git clone ${GITLAB_URL}/software-factory/etx_app_base_app.git
cd etx_app_base_app
```

The Git repository is named `etx_app_base_app`. Kubernetes resources created later in the workshop use the naming convention `etx-app-base-app-*`.

This repository intentionally does **not** contain a `devfile.yaml`. In the next lab, you'll create one yourself and use it to launch a Dev Spaces workspace.

Verify OpenShift Access

Confirm that you're connected to the openshift cluster and have access to the development project.

```
oc whoami --show-server

oc project etx-app-dev
```

Lab 2

Overview

In this lab, you'll deploy the shared platform services that support the Application Platform Software Factory.

These services provide the database, messaging, CI/CD, and observability capabilities required by the application throughout the remainder of the lab.

By the end of this lab, you'll have a working development platform that can be used by Dev Spaces, OpenShift Pipelines, and GitOps deployments.

HashiCorp Vault (Pre-Deployed)

HashiCorp Vault provides centralized secrets management for the Application Platform Software Factory. It securely stores sensitive information such as passwords, API keys, tokens, and registry credentials, allowing applications and CI/CD pipelines to retrieve secrets at runtime instead of storing them in source code or configuration files.

The Vault instance has already been deployed and configured as part of the workshop infrastructure.

Property	Value
URL	Refer to the **RHDP Lab Information** page.
Authentication	OIDC (Sign in using your workshop credentials)
Supported Users	`platform-admin`, `platform-developer`
Purpose	Secure storage of application secrets and CI/CD credentials

Throughout this workshop, OpenShift Pipelines will securely retrieve credentials from Vault whenever they are required to access external services such as container registries.

Open the Vault UI using the URL provided in the **RHDP Lab Information** page and sign in using your workshop credentials. If prompted for a role during login, leave

> the field blank unless instructed otherwise.

Install Red Hat OpenShift Pipelines

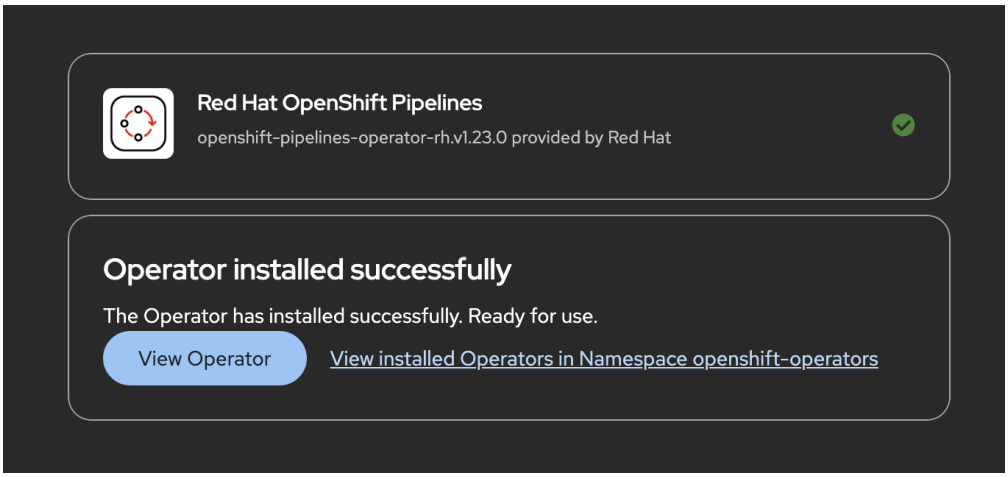
Red Hat OpenShift Pipelines is a Kubernetes-native CI/CD solution based on the open source **Tekton** project. It automates common software delivery tasks such as building applications, running tests, creating container images, and publishing artifacts.

Throughout this lab, you'll use OpenShift Pipelines to automate the application's build process before deploying it through GitOps.

Install the Operator

Install the Red Hat OpenShift Pipelines Operator from OperatorHub.

1. Open the OpenShift Web Console.
2. Log in using the `admin` credentials provided on the RHDP lab information page.
3. From the navigation menu, select menu:Ecosystem[OperatorHub].
4. Search for **Red Hat OpenShift Pipelines**. <it might take sometime to load the operator, please wait>
5. Select the operator and click btn:[Install].
6. Accept the default installation settings:
   - **Update channel:** `latest`
   - **Installation mode:** `All namespaces on the cluster`
   - **Installed namespace:** `openshift-operators`
7. Click btn:[Install].
8. Wait until the operator status changes to **Succeeded**.



Verify the Installation

Verify that the operator has been installed successfully.

```
oc get csv -n openshift-operators | grep pipelines
```

The command should return the OpenShift Pipelines ClusterServiceVersion (CSV) with a status of `Succeeded`.

Next, verify that the Tekton platform components are running.

```
oc get pods -n openshift-pipelines
```

You should see the Tekton controller pods in the `Running` state, including:

- `tekton-pipelines-controller`
- `tekton-triggers-controller`
- `tekton-events-controller`
- `tekton-webhook`

Installing the operator automatically deploys the Tekton controllers and supporting components into the `openshift-pipelines` namespace. These components will execute the CI/CD pipelines you create later in this lab.

Install Red Hat AMQ Streams

Red Hat AMQ Streams provides Apache Kafka as a managed service on OpenShift. Kafka enables applications to exchange messages asynchronously, making it easier to build scalable, event-driven applications.

In this workshop, the ETX App Base application uses Kafka to exchange messages between application services.

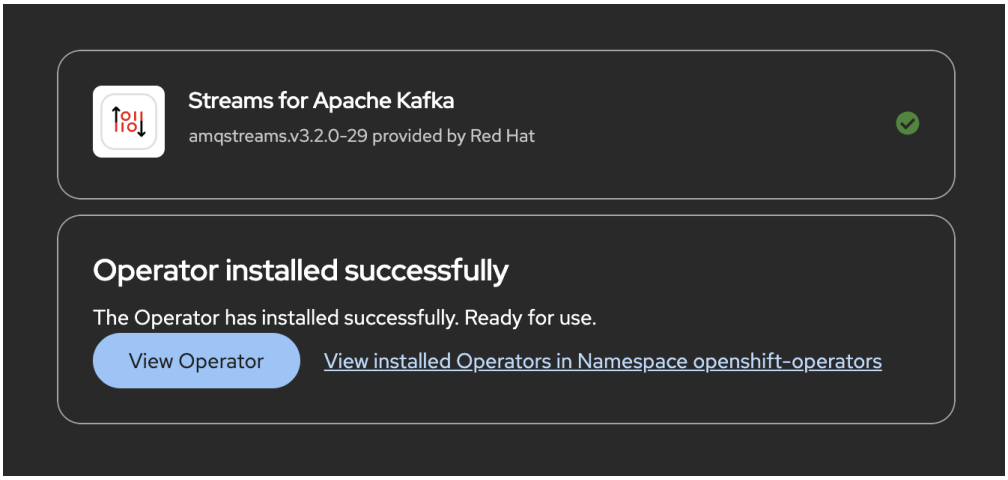
Install the Operator

1. Open the OpenShift Web Console.
2. From the navigation menu, select menu:Operators[OperatorHub].
3. Search for **Streams for Apache Kafka**.

Install **Streams for Apache Kafka** only. Do **not** install **Streams for Apache Kafka Proxy**.

4. Click btn:[Install].
5. Accept the default installation settings:
    - **Update channel:** `stable`
    - **Installation mode:** `All namespaces`
    - **Installed namespace:** `openshift-operators`

6. Click btn:[Install].
7. Wait until the operator status changes to **Succeeded**.



Verify the Installation

Verify that the operator has been installed successfully.

```
oc get csv -n openshift-operators | grep amqstreams
```

The output should show the AMQ Streams ClusterServiceVersion (CSV) in the `Succeeded` state.

Deploy a Kafka Cluster

With the operator installed, you can now deploy a Kafka cluster in your application namespace.

Starting with AMQ Streams 3.x, Kafka clusters use **KafkaNodePools** to manage broker nodes.

A KafkaNodePool defines a group of Kafka nodes, their roles, and storage configuration. In this workshop, you'll deploy a single-node Kafka cluster running in **KRaft mode**, where the same node acts as both the controller and broker. This removes the need for ZooKeeper and simplifies the deployment for development environments.

Apply the following manifest:

```
oc apply -f - <<'EOF'
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool
metadata:
  name: kafka
  namespace: etx-app-dev
  labels:
    strimzi.io/cluster: parasol-kafka
spec:
  replicas: 1
  roles:
    - controller
```

```
    - broker
  storage:
    type: ephemeral
---
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: parasol-kafka
  namespace: etx-app-dev
  annotations:
    strimzi.io/node-pools: enabled
    strimzi.io/kraft: enabled
spec:
  kafka:
    version: 4.2.0
    metadataVersion: 4.2-IV0
    listeners:
      - name: plain
        port: 9092
        type: internal
        tls: false
      - name: tls
        port: 9093
        type: internal
        tls: true
    config:
      offsets.topic.replication.factor: 1
      transaction.state.log.replication.factor: 1
      transaction.state.log.min.isr: 1
      default.replication.factor: 1
      min.insync.replicas: 1
  entityOperator:
    topicOperator: {}
    userOperator: {}
EOF
```

Verify the Kafka Cluster

Wait for the Kafka cluster to become ready.

```
oc wait kafka/parasol-kafka \
  --for=condition=Ready \
  --timeout=300s \
  -n etx-app-dev
```

Verify that the Kafka pods are running.

```
oc get pods \
  -l strimzi.io/cluster=parasol-kafka \
```

```
  -n etx-app-dev
```

The output should include:

- `parasol-kafka-kafka-0`
- `parasol-kafka-entity-operator-*`

Ensure all Kafka pods are in the `Running` state before proceeding.

Verify the Bootstrap Service

Applications connect to Kafka through the bootstrap service.

Retrieve the service endpoint.

```
oc get service parasol-kafka-kafka-bootstrap \
  -n etx-app-dev \
  -o jsonpath='{.metadata.name}:{.spec.ports[0].port}{"\n"}'
```

The command should return:

```
parasol-kafka-kafka-bootstrap:9092
```

This bootstrap endpoint will be used later when configuring the application.

Deploy PostgreSQL Database

Deploy PostgreSQL using the Red Hat certified container image for application data persistence.

Create PostgreSQL Deployment

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: Secret
metadata:
  name: parasol-db-secret
  namespace: etx-app-dev
type: Opaque
stringData:
  database-name: parasol
  database-user: parasol
  database-password: parasol
---
apiVersion: v1
kind: Service
metadata:
  name: parasol-db
```

```
  namespace: etx-app-dev
spec:
  ports:
    - name: postgresql
      port: 5432
      targetPort: 5432
  selector:
    app: parasol-db
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: parasol-db
  namespace: etx-app-dev
spec:
  replicas: 1
  selector:
    matchLabels:
      app: parasol-db
  template:
    metadata:
      labels:
        app: parasol-db
    spec:
      containers:
        - name: postgresql
          image: registry.redhat.io/rhel9/postgresql-16:latest
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRESQL_USER
              valueFrom:
                secretKeyRef:
                  name: parasol-db-secret
                  key: database-user
            - name: POSTGRESQL_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: parasol-db-secret
                  key: database-password
            - name: POSTGRESQL_DATABASE
              valueFrom:
                secretKeyRef:
                  name: parasol-db-secret
                  key: database-name
          volumeMounts:
            - name: postgresql-data
              mountPath: /var/lib/pgsql/data
          livenessProbe:
            exec:
              command:
```

```
                - /usr/libexec/check-container
                - --live
            initialDelaySeconds: 120
            timeoutSeconds: 10
          readinessProbe:
            exec:
              command:
                - /usr/libexec/check-container
            initialDelaySeconds: 5
            timeoutSeconds: 1
          resources:
            limits:
              memory: 512Mi
              cpu: 500m
            requests:
              memory: 256Mi
              cpu: 100m
      volumes:
        - name: postgresql-data
          emptyDir: {}
EOF
```

This Deployment uses `emptyDir` for storage, which means data is lost when the pod restarts. For production, use a `PersistentVolumeClaim` backed by persistent storage.

Wait for PostgreSQL to be ready:

```
oc rollout status deployment/parasol-db -n etx-app-dev
```

Verify the PostgreSQL pod is running:

```
oc get pods -l app=parasol-db -n etx-app-dev
```

Test the database connection:

```
oc run postgresql-test --rm -it --restart=Never \
  --image=registry.redhat.io/rhel9/postgresql-16:latest \
  --env PGPASSWORD=$(oc get secret -n etx-app-dev parasol-db-secret -o
jsonpath='{.data.database-password}' | base64 -d) \
  -n etx-app-dev -- \
  psql -h parasol-db \
  -U $(oc get secret -n etx-app-dev parasol-db-secret -o jsonpath='{.data.database-
user}' | base64 -d) \
  -d $(oc get secret -n etx-app-dev parasol-db-secret -o jsonpath='{.data.database-
name}' | base64 -d) \
  -c '\conninfo'
```

You should see connection information confirming PostgreSQL is accessible.

The database credentials are stored in the `parasol-db-secret` Secret. Your application will use these same credentials to connect.

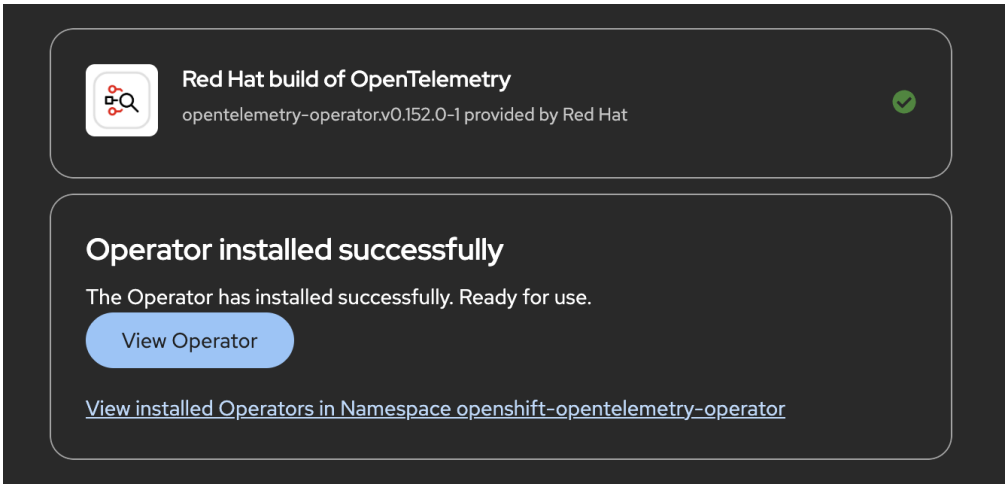
Install the Red Hat Build of OpenTelemetry

The Red Hat Build of OpenTelemetry provides the observability foundation for cloud-native applications. It enables applications to collect and export telemetry data such as distributed traces, metrics, and logs.

In this workshop, you'll install the OpenTelemetry Operator, deploy an OpenTelemetry Collector, and configure automatic instrumentation for Java applications.

Install the Operator

1. Open the OpenShift Web Console.
2. From the navigation menu, select menu:Operators[OperatorHub].
3. Search for **Red Hat build of OpenTelemetry**.
4. Click btn:[Install].
5. Accept the default installation settings:
   - **Update channel:** `stable`
   - **Installation mode:** `All namespaces`
   - **Installed namespace:** `openshift-opentelemetry-operator`
6. Click btn:[Install].
7. Wait until the operator status changes to **Succeeded**.



Verify the Installation

Verify that the operator has been installed successfully.

```
oc get csv -n openshift-opentelemetry-operator
```

The output should show the OpenTelemetry Operator ClusterServiceVersion (CSV) in the `Succeeded` state.

Deploy an OpenTelemetry Collector

An OpenTelemetry Collector receives telemetry data from instrumented applications and forwards it to one or more observability backends.

For this workshop, you'll deploy a collector that exports traces to its own logs for demonstration purposes.

Apply the following manifest:

```
oc apply -f - <<'EOF'
apiVersion: opentelemetry.io/v1beta1
kind: OpenTelemetryCollector
metadata:
  name: otel-collector
  namespace: etx-app-dev
spec:
  mode: deployment
  config:
    receivers:
      otlp:
        protocols:
          grpc:
            endpoint: 0.0.0.0:4317
          http:
            endpoint: 0.0.0.0:4318
    processors:
      batch: {}
      probabilistic_sampler:
        sampling_percentage: 100.0
    exporters:
      debug:
        verbosity: detailed
    service:
      pipelines:
        traces:
          receivers: [otlp]
          processors: [probabilistic_sampler, batch]
          exporters: [debug]
EOF
```

The `debug` exporter writes trace information to the collector logs. It is suitable for demonstrations and development environments.

In production, replace the `debug` exporter with a tracing backend such as Red Hat

build of Tempo, Jaeger, or another supported OpenTelemetry-compatible platform.

Verify the Collector

Wait for the collector deployment to become available.

```
oc rollout status deployment/otel-collector-collector \
  -n etx-app-dev
```

Verify that the collector pod is running.

```
oc get pods \
  -l app.kubernetes.io/name=otel-collector-collector \
  -n etx-app-dev
```

Ensure the collector pod is in the `Running` state before continuing.

Configure Automatic Instrumentation

The OpenTelemetry Operator can automatically instrument Java applications without modifying application code.

Create an `Instrumentation` resource that defines how Java applications should export telemetry.

Apply the following manifest:

```
oc apply -f - <<'EOF'
apiVersion: opentelemetry.io/v1alpha1
kind: Instrumentation
metadata:
  name: java-instrumentation
  namespace: etx-app-dev
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
spec:
  exporter:
    endpoint: http://otel-collector-collector:4317
  propagators:
    - tracecontext
    - baggage
  sampler:
    type: always_on
  java:
    image: ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-
java:latest
EOF
```

The "instrumentation.opentelemetry.io/inject-java: true" enable automatic instrumentation.

When the application is deployed, the OpenTelemetry Operator automatically injects the Java agent into the application container.

The `always_on` sampler captures every trace generated by the application, making it ideal for workshop environments. Production environments typically use sampling strategies to reduce telemetry volume.

Verify Automatic Instrumentation

After deploying an application with the instrumentation annotation, verify that traces are being received by the collector.

You can view the collector logs using:

```
oc logs deployment/otel-collector-collector -n etx-app-dev
```

You should see trace data being exported by the `debug` exporter.

You'll use this configuration later in the workshop when deploying the ETX App Base application.

Summary

You now have the following services deployed and ready:

- **Vault** (pre-deployed) — secrets management with Kubernetes authentication for pipeline access
- **OpenShift Pipelines** — Tekton CI engine ready to run pipelines
- **AMQ Streams Kafka Cluster** — `parasol-kafka-kafka-bootstrap:9092` with `intake` topic
- **PostgreSQL Database** — `parasol-db:5432` with database `parasol`
- **OpenTelemetry** — collector and auto-instrumentation configured for distributed tracing

Your application can now connect to these services:

- **Kafka bootstrap server:** `parasol-kafka-kafka-bootstrap:9092`
- **PostgreSQL host:** `parasol-db:5432`
- **Database credentials:** Available in `parasol-db-secret` Secret

DevEx and DevSpaces

Developer Experience Matters

Why Developer Experience Matters

- Modern software development requires developers to work with numerous tools throughout the Software Development Lifecycle (SDLC).
- A typical developer interacts with:
  - Source code repositories
  - Development environments
  - CI/CD pipelines
  - GitOps deployment tools
  - Container registries
  - Documentation portals
  - Security scanners
  - Monitoring and observability platforms

- Each tool solves a specific problem. However, switching between multiple tools increases the amount of information developers must understand before they can deliver software.
- This additional mental effort is commonly referred to as **developer cognitive load**.
- Platform Engineering aims to reduce this cognitive load by providing standardized workflows and self-service capabilities, allowing developers to spend more time writing applications and less time managing platform complexity.

Great platforms are not measured by the number of tools they provide, but by how simple they make software delivery for developers.

What is Developer Experience?

- *Developer Experience (DevEx)* describes the overall experience developers have while building, testing, deploying, and operating software.
- It encompasses the tools, workflows, platforms, environments, and organizational practices that influence how efficiently developers can deliver applications.

> Developer Experience is the user experience developers have while interacting with a platform, its tools, workflows, and supporting services.

- The primary goals of DevEx are to:
  - Reduce friction during software development.
  - Improve developer productivity.
  - Standardize development workflows.
  - Enable developers to focus on delivering business value.

Benefits of Improving Developer Experience

Improving Developer Experience benefits both developers and organizations.

Technical Benefits

- Reduced cognitive load
- Standardized development workflows
- Faster onboarding of new developers
- Faster software delivery
- Improved software quality
- Consistent engineering practices

Business Benefits

- Higher developer productivity

- Faster time to market
- Lower software development costs
- Reduced developer burnout
- Higher employee retention
- Increased innovation

Organizations that invest in Developer Experience often observe improvements in software delivery performance, developer satisfaction, and overall business outcomes.

Developer Experience and Platform Engineering

Developer Experience is closely related to several Platform Engineering concepts.

Platform Engineering

Builds, operates, and maintains the internal platform that enables developers to build and deliver software efficiently.

Internal Developer Platform (IDP)

A collection of standardized tools, automation, services, and workflows that developers use throughout the Software Development Lifecycle.

Developer Experience (DevEx)

Measures how effectively developers can use the Internal Developer Platform to deliver software.

Internal Developer Portal

A centralized, self-service interface where developers discover services, documentation, templates, and workflows required to build and operate applications.

Dev Spaces Overview

In the previous lesson, you learned how a **Software Factory** standardizes software delivery by providing developers with reusable services, automation, and engineering best practices.

One of the first capabilities provided by a Software Factory is a standardized development environment.

Instead of asking every developer to install IDEs, programming language runtimes, SDKs, CLI tools, and dependencies on their own workstation, organizations provide ready-to-use development environments that developers can access on demand.

This capability is provided by **Red Hat OpenShift Dev Spaces**.

Think about how long it takes today for a new developer to prepare their laptop before writing their first line of code. How much of that effort could be eliminated with a preconfigured development environment?

The Challenge

Setting up a local development environment is often one of the most time-consuming parts of developer onboarding.

A typical setup includes:

- Installing an IDE.
- Installing programming language runtimes.
- Downloading SDKs and command-line tools.
- Configuring Kubernetes access.
- Installing extensions and plugins.
- Managing project dependencies.
- Matching software versions used by the rest of the team.

Even after completing this setup, developers frequently encounter environment differences that lead to the familiar problem:

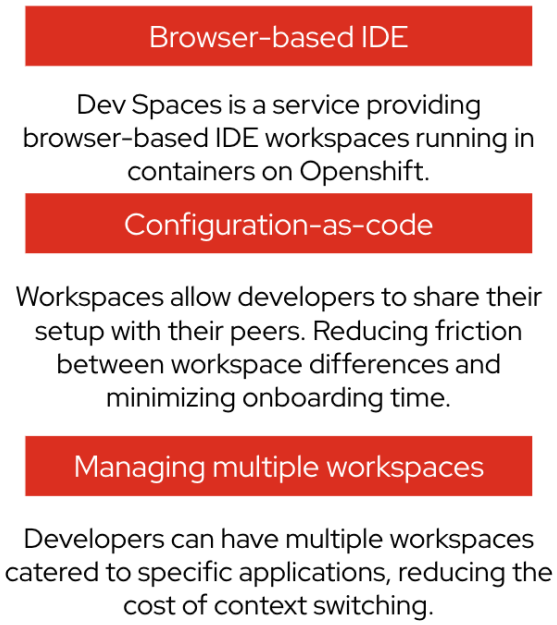
> "It works on my machine."

These inconsistencies slow onboarding, complicate collaboration, and introduce avoidable deployment issues.

When every developer maintains a different workstation configuration, development environments gradually drift apart, making troubleshooting, collaboration, and onboarding increasingly difficult.

What is OpenShift Dev Spaces?

OpenShift Dev Spaces provides **Cloud Development Environments (CDEs)** that run directly on OpenShift.



Instead of developing on a local workstation, developers use browser-based workspaces that are already configured with the required IDE, programming language runtimes, tools, dependencies, and cluster access.

Each workspace runs as containers on the OpenShift cluster while developers access it through a web browser using a familiar development environment.

This allows every developer to work with a consistent and reproducible environment regardless of the machine they are using.

Why Cloud Development Environments?

Cloud Development Environments improve both developer productivity and platform consistency.

Capability	Benefit
Rapid onboarding	Developers begin coding within minutes instead of spending hours preparing their workstation.
Consistent environments	Every developer uses the same tools, runtime versions, and dependencies.
Cloud-native development	Applications are developed in an environment that closely matches production.
Centralized security	Source code and development environments remain inside the organization's OpenShift platform.
Platform management	Development environments are centrally maintained by Platform Engineers.

Dev Spaces in the Software Factory

Within a Software Factory, Dev Spaces provides the standardized environment where application development begins.

A typical workflow looks like this:

1. Launch a development workspace.
2. Clone or open the application source code.
3. Develop and test application features.
4. Commit changes to Git.
5. Trigger the automated software delivery pipeline.
6. Allow the Software Factory to build, validate, and deploy the application.

Developers focus on writing application code while the Software Factory handles the surrounding automation and platform operations.

Benefits

For developers:

- Start coding immediately without local setup.
- Access projects from any machine using only a browser.
- Work in consistent environments across development teams.
- Easily switch between projects without reinstalling tools.

For Platform Engineers:

- Standardize development environments.
- Simplify developer onboarding.
- Improve security by centralizing development resources.
- Reduce environment-related support requests.

OpenShift Dev Spaces standardizes where developers build software, allowing Platform Engineers to centrally manage development environments while developers focus on delivering application features.

Reflection

▼ *Think about your own organization (Click to expand)*

- How long does it take to prepare a new developer workstation?
- Which tools must developers install manually?
- How often do environment differences cause development or deployment issues?
- How could centralized development environments improve developer productivity?

Check Your Understanding

▼ *Quiz Time! (Click to expand)*

Which statement best describes OpenShift Dev Spaces?

1. A desktop IDE installed on developer laptops.
2. A cloud-based development environment running on OpenShift that provides standardized workspaces.
3. A Continuous Integration platform.
4. A Git repository hosting service.

Answer

B — OpenShift Dev Spaces provides standardized, browser-based development environments running on OpenShift.

Key Takeaway

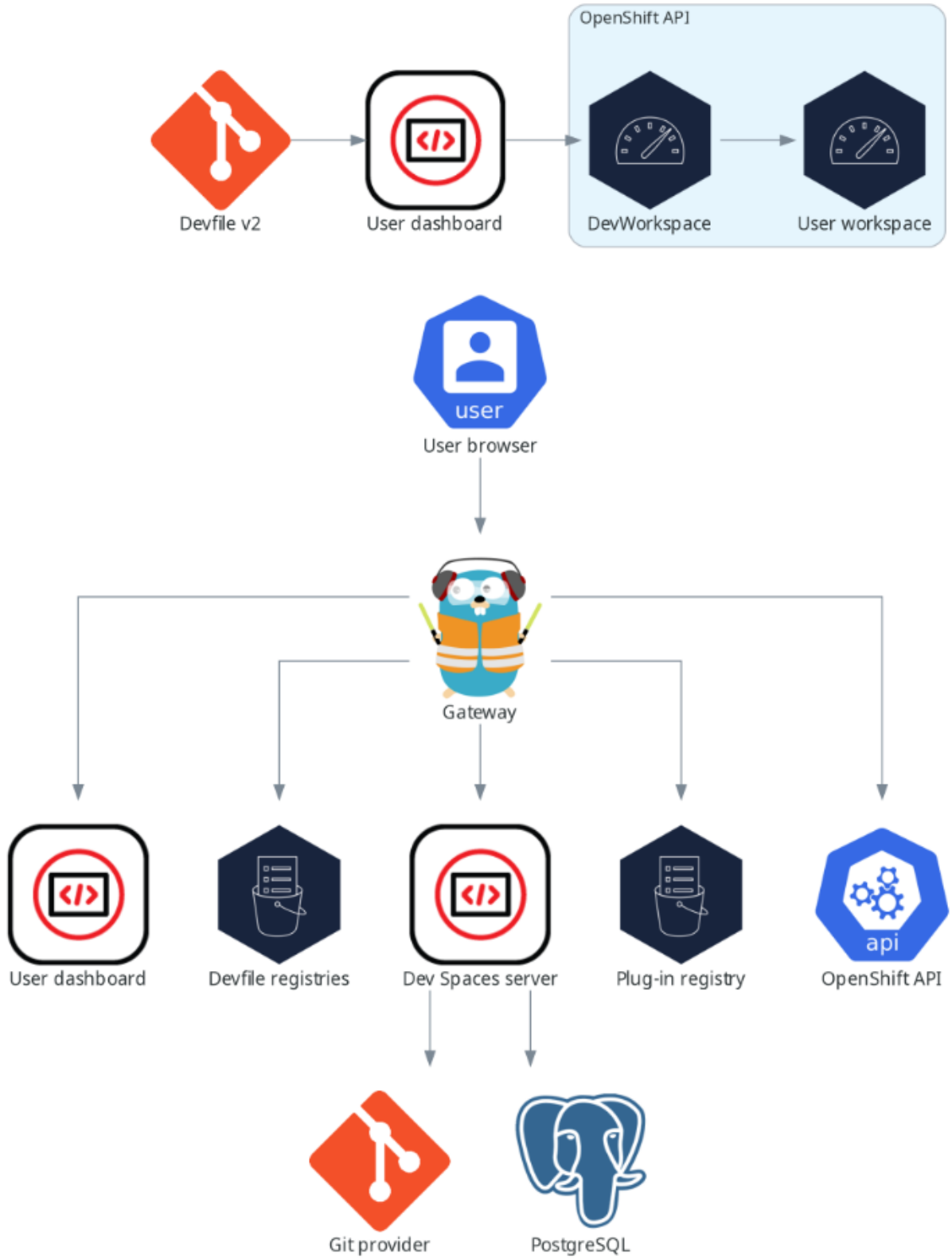
OpenShift Dev Spaces provides standardized Cloud Development Environments that eliminate workstation setup, improve consistency, and enable developers to start building applications quickly within the Software Factory.

Dev Spaces Architecture

In the previous lesson, you learned how OpenShift Dev Spaces provides standardized Cloud Development Environments for developers.

In this lesson, you'll look behind the scenes to understand how Dev Spaces creates and manages developer workspaces on OpenShift.

You don't need to manage these components directly, but understanding their roles will help you see how Dev Spaces delivers secure, repeatable development environments.



How a Workspace is Created

When a developer creates a new workspace, Dev Spaces follows a simple workflow:

1. The developer selects or imports a project that contains a **Devfile**.
2. The Dev Spaces Dashboard sends the workspace request.

3. The Dev Spaces Server reads the Devfile definition.
4. A **DevWorkspace** resource is created on OpenShift.
5. OpenShift provisions the required workspace containers.
6. The developer connects to the running workspace through the browser.

This process ensures every workspace is created from the same version-controlled configuration, providing a consistent development environment for every developer.

Core Components

The architecture consists of several services that work together to provision and manage developer workspaces.

Component	Purpose
Devfile Registry	Stores reusable Devfile templates that define workspace configurations and sample development environments.
Plugin Registry	Provides IDE extensions and plugins that are installed into developer workspaces.
Dev Spaces Dashboard	The browser-based interface where developers create, start, stop, and manage their workspaces.
Dev Spaces Server	The core service that processes workspace requests, interprets Devfiles, and coordinates workspace creation.
Gateway	Routes user requests to the appropriate Dev Spaces services and workspace pods. It also handles authentication and authorization using OpenShift OAuth and Role-Based Access Control (RBAC).
PostgreSQL	Stores configuration data and workspace metadata used by the Dev Spaces platform.
OpenShift API	Creates and manages the Kubernetes resources required for each developer workspace.
DevWorkspace	A Kubernetes custom resource that represents an individual development workspace running on OpenShift.

How Dev Spaces Fits the Software Factory

Within the Software Factory, Dev Spaces provides the standardized development environment where application development begins.

Platform Engineers maintain the Dev Spaces platform, while developers simply launch a workspace and begin writing code.

Because workspaces are defined as code and provisioned automatically, organizations gain:

- Consistent development environments
- Faster developer onboarding

- Simplified platform management
- Improved security and governance

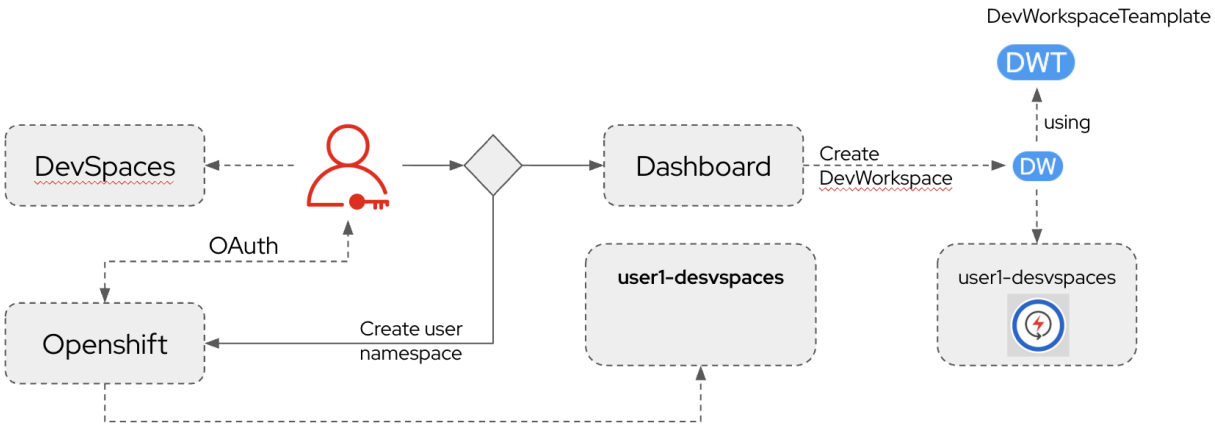
Most developers interact only with the Dev Spaces Dashboard. The remaining components operate behind the scenes to automatically provision, secure, and manage development workspaces on OpenShift.

Key Takeaway

OpenShift Dev Spaces combines Devfiles, platform services, and OpenShift to automatically provision standardized development workspaces, allowing developers to focus on building applications instead of configuring their development environment.

Creating a Workspace

Now let's see what actually happens when a developer signs in and creates a development workspace.



Workspace Creation Workflow

Creating a workspace is designed to be a simple self-service experience.

Behind the scenes, Dev Spaces automatically provisions the required OpenShift resources so developers can begin coding within minutes.

The workflow consists of the following steps:

1. The developer signs in to the Dev Spaces Dashboard using their OpenShift credentials.
2. Dev Spaces authenticates the user through OpenShift OAuth.
3. If this is the user's first login, OpenShift automatically creates a dedicated namespace named `<username>-devspaces`.
4. The developer creates either:

    - An empty workspace, or
    - A workspace based on a predefined template.
5. Dev Spaces creates a **DevWorkspace** resource.
6. The DevWorkspace references a **DevWorkspaceTemplate**, which defines the containers, tools, storage, and runtime configuration required for the workspace.
7. OpenShift provisions the workspace pod.
8. The developer connects to the running workspace through their browser and begins coding.

Authentication

By default, OpenShift Dev Spaces integrates directly with the OpenShift authentication system.

This provides a seamless sign-in experience using existing cluster credentials.

Key points:

- Uses OpenShift OAuth by default.
- Supports any OpenID Connect (OIDC) identity provider.
- Can integrate with Red Hat Single Sign-On (RHSSO) or Red Hat Build of Keycloak.
- Uses OpenShift Role-Based Access Control (RBAC) to control access to workspaces.

Because authentication is delegated to OpenShift, developers don't need separate Dev Spaces accounts.

Automatic Workspace Isolation

Every developer receives an isolated workspace namespace.

For example:

```
alice-devspaces
bob-devspaces
charlie-devspaces
```

Each namespace contains only that developer's workspaces and resources.

This provides:

- Resource isolation
- Better security
- Simplified administration
- Multi-tenant development environments

Platform Engineers can centrally manage resource quotas and permissions while developers remain isolated from one another.

Workspace Templates

Developers don’t need to manually configure development environments.

Instead, they create workspaces from reusable templates.

A template typically defines:

- Development container image
- IDE configuration
- Required tools and runtimes
- Persistent storage
- Environment variables
- Startup commands

These templates ensure every developer starts from a consistent environment that follows organizational standards.

Developers simply choose a template and start coding. Dev Spaces automatically provisions the required OpenShift resources, creates the development workspace, and provides browser-based access to a fully configured environment.

Key Takeaway

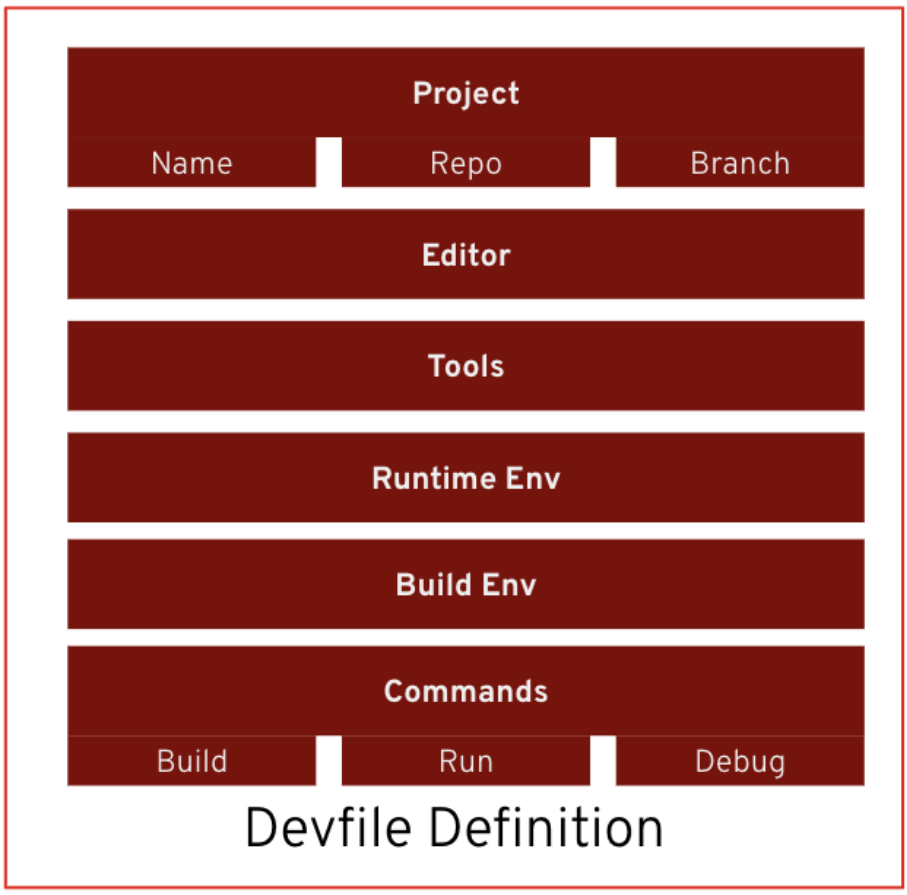
OpenShift Dev Spaces automates workspace provisioning by authenticating users through OpenShift, creating isolated namespaces, and provisioning standardized development environments from reusable workspace templates.

Devfiles

In the previous lesson, you learned how OpenShift Dev Spaces provisions developer workspaces automatically.

But how does Dev Spaces know which tools, runtimes, and commands should be available inside each workspace?

The answer is a **Devfile**.



What is a Devfile?

A Devfile is a YAML file, typically named `devfile.yaml`, stored in the root of a Git repository.

Platform Engineers use Devfiles to describe and standardize development environments, allowing every developer working on a project to use the same tools, runtime, and workspace configuration.

Instead of manually configuring development environments, Dev Spaces simply reads the Devfile and provisions the workspace automatically.

A Devfile acts as the blueprint for a development workspace. It defines everything needed to create a consistent development environment.

What Does a Devfile Define?

A Devfile can describe every aspect of a development workspace, including:

- Project metadata such as the repository and default branch
- The development editor and IDE configuration
- Container images and runtime environment
- Development tools and dependencies
- Build and runtime environments
- Volumes and persistent storage

- Build, run, debug, and test commands

Because the Devfile is version-controlled alongside the application source code, every developer receives the same development environment regardless of their local machine.

Why Devfiles Matter

Without Devfiles, every developer must configure their own workstation.

This often leads to inconsistent tool versions, missing dependencies, and the classic problem:

> "It works on my machine."

Devfiles eliminate this by defining the workspace as code.

Benefits include:

- Consistent development environments
- Faster developer onboarding
- Reproducible workspaces
- Version-controlled environment definitions
- Simplified collaboration across teams

Devfile Structure

A Devfile is written in YAML and is organized into several logical sections.

```
schemaVersion: 2.2.0
metadata:
  name: my-project

attributes: {}   # IDE settings
components: []   # Containers, volumes, endpoints
commands: []     # Build, run, debug commands
events: {}       # Lifecycle hooks
```

The most common sections are:

Section	Purpose
Metadata	Identifies the workspace and project.
Attributes	Stores IDE-specific configuration, such as recommended extensions.
Components	Defines containers, volumes, endpoints, and other workspace resources.
Commands	Specifies build, run, test, and debug commands available from the IDE.
Events	Executes lifecycle actions such as `preStart` and `postStart`.

Devfiles and the Universal Developer Image

A Devfile is recommended, but it is not required to create a workspace.

If a Git repository does not contain a `devfile.yaml`, OpenShift Dev Spaces automatically provisions a workspace using the **Universal Developer Image (UDI)**.

The Universal Developer Image includes many commonly used development tools and programming language runtimes, allowing developers to start coding immediately.

As projects become more specialized, teams typically provide their own Devfile to customize the workspace with project-specific tools, dependencies, and commands.

How Devfiles Fit the Software Factory

Within the Software Factory, Platform Engineers maintain Devfiles as reusable definitions for development environments.

Developers simply clone a repository or create a new workspace, and Dev Spaces provisions the environment automatically.

This ensures every project follows organizational standards while reducing setup time and operational overhead.

Devfiles define development environments as code, allowing organizations to deliver standardized, repeatable, and version-controlled workspaces across teams.

Key Takeaway

A Devfile is the blueprint for a Dev Spaces workspace. It defines the tools, containers, commands, and runtime configuration required for development, enabling consistent and reproducible environments for every developer.

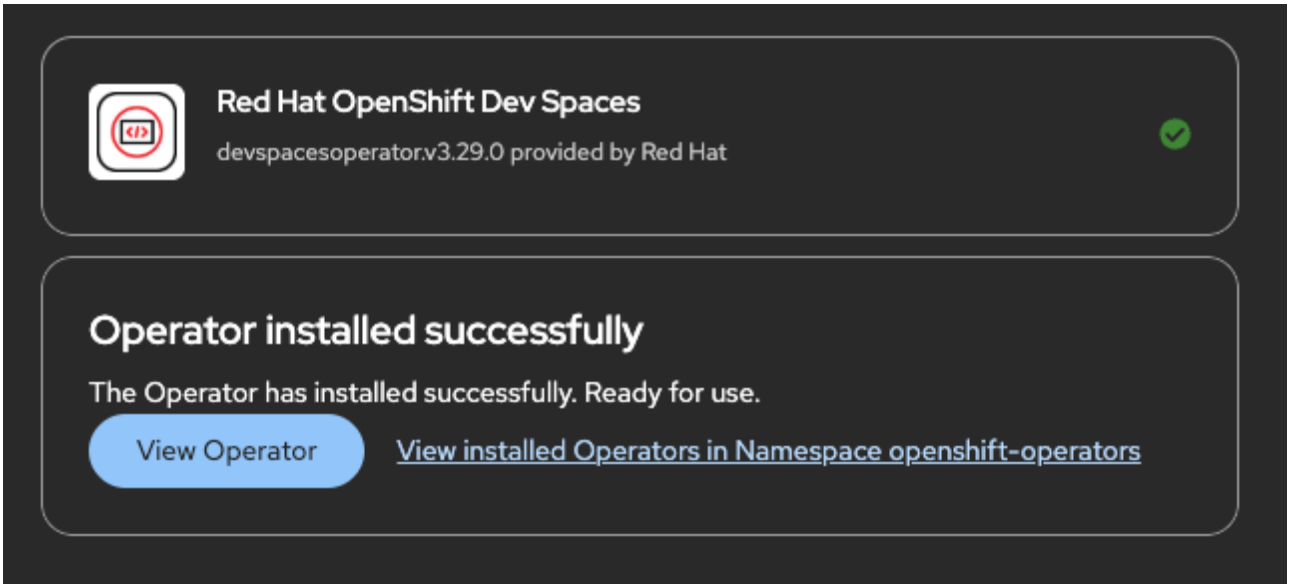
Hands-on Lab

Lab 3

Install the Dev Spaces Operator

Install via OperatorHub

1. In the OpenShift Web Console, go to menu:Ecosystem[Software Catalog]
2. Search for **Red Hat OpenShift Dev Spaces**
3. Click btn:[Install]
4. Keep the operator installation namespace as `openshift-operators`
5. Accept the defaults for the rest (All namespaces, Automatic approval) and click btn:[Install]
6. Wait for the operator to reach the `Succeeded` phase



Verify from the CLI:

```
oc get csv -n openshift-operators | grep devspaces
```

A similar output should be displayed:

```
devspacesoperator.v3.27.1          Red Hat OpenShift Dev Spaces   3.27.1
devspacesoperator.v3.27.0          Succeeded
```

The DevWorkspace Operator is installed automatically as a dependency.

The operator runs from `openshift-operators`. The `openshift-devspaces` namespace will be created in the next step for the `CheCluster` instance and Dev Spaces server

components.

Lab 4

<Add this as a separate section from the lab>

Create the CheCluster

The `CheCluster` custom resource is the single configuration object for your Dev Spaces deployment. This is the sample configuration to create an instance with sensible defaults for this workshop environment:

```
apiVersion: org.eclipse.che/v2
kind: CheCluster
metadata:
  name: devspaces
  namespace: openshift-devspaces
spec:
  components:
    cheServer:
      debug: false
      logLevel: INFO
    dashboard: {}
    devWorkspace: {}
    devfileRegistry: {}
    imagePuller:
      enable: false # ①
    metrics:
      enable: true
    pluginRegistry:
      openVSXURL: "https://open-vsx.org" # ②
  containerRegistry: {}
  devEnvironments:
    startTimeoutSeconds: 600
    secondsOfRunBeforeIdling: -1 # ③
    secondsOfInactivityBeforeIdling: 1800
    maxNumberOfWorkspacesPerUser: 5
    maxNumberOfRunningWorkspacesPerUser: 2
    defaultContainerResources:
      limits:
        cpu: "2"
        memory: 6Gi
      requests:
        cpu: 500m
        memory: 1Gi
    disableContainerRunCapabilities: false # ④
    defaultEditor: che-incubator/che-code/latest
    defaultNamespace:
      autoProvision: true
```

```
      template: <username>-devspaces
    storage:
      pvcStrategy: per-workspace # ⑤
```

① Image puller pre-caches images on nodes to speed up workspace startup. Disable for small clusters; enable if startup time matters.

② Points at the public Open VSX registry so developers can install any extension. The default embedded registry has a very limited set.

③ Set to `-1` to disable run-time idling (useful during workshops). In production, set a reasonable value.

④ Required for nested containers (Podman-in-workspace). Must be `false`.

⑤ `per-workspace` gives each workspace its own PVC. This supports multiple workspaces per user without conflicts and provides clean isolation. Alternative: `per-user` shares a single PVC across all workspaces.

Create the namespace for Dev Spaces:

```
oc create namespace openshift-devspaces
```

Apply the CheCluster configuration:

```
oc apply -f - <<'EOF'
apiVersion: org.eclipse.che/v2
kind: CheCluster
metadata:
  name: devspaces
  namespace: openshift-devspaces
spec:
  components:
    cheServer:
      debug: false
      logLevel: INFO
    dashboard: {}
    devWorkspace: {}
    devfileRegistry: {}
    imagePuller:
      enable: false
    metrics:
      enable: true
    pluginRegistry:
      openVSXURL: "https://open-vsx.org"
  containerRegistry: {}
  devEnvironments:
    startTimeoutSeconds: 600
    secondsOfRunBeforeIdling: -1
    secondsOfInactivityBeforeIdling: 1800
    maxNumberOfWorkspacesPerUser: 5
```

```
    maxNumberOfRunningWorkspacesPerUser: 2
    defaultContainerResources:
      limits:
        cpu: "2"
        memory: 6Gi
      requests:
        cpu: 500m
        memory: 1Gi
    disableContainerRunCapabilities: false
    defaultEditor: che-incubator/che-code/latest
    defaultNamespace:
      autoProvision: true
      template: <username>-devspaces
    storage:
      pvcStrategy: per-workspace
EOF
```

Dev Spaces on OpenShift uses OpenShift OAuth for authentication by default. The Dev Spaces operator is designed to integrate with OpenShift's built-in authentication system, not with external OIDC providers like Keycloak. Users will authenticate with their OpenShift credentials when accessing the Dev Spaces dashboard.

Wait for the deployment to complete.

```
oc get checluster -n openshift-devspaces
```

You should see a resource named `devspaces`.

```
oc get pods -n openshift-devspaces -w
```

Wait until all pods are in `Running` state with `READY` showing the expected number of containers (e.g., `1/1`, `4/4`).

Once all pods are running, check the CheCluster status:

```
oc get checluster devspaces -n openshift-devspaces -o
jsonpath='{.status.chePhase}{"\n"}'
```

The output should be `Active`.

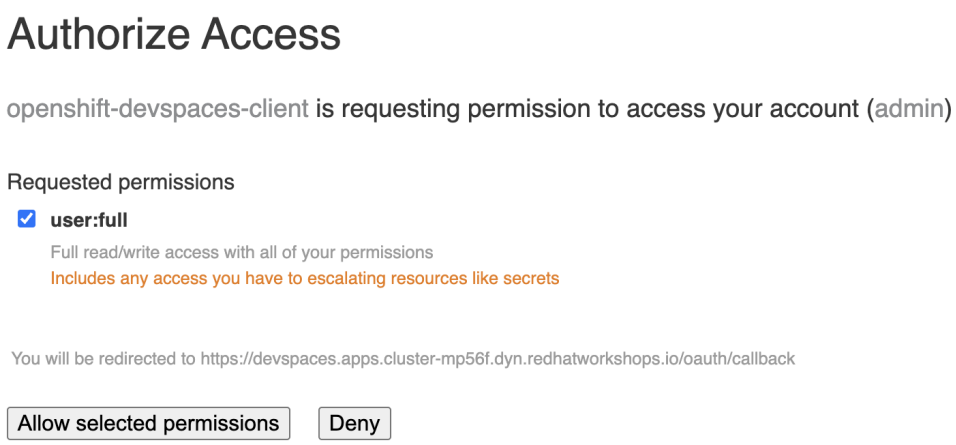
Access the Dashboard

Once the CheCluster is `Active`, get the dashboard URL:

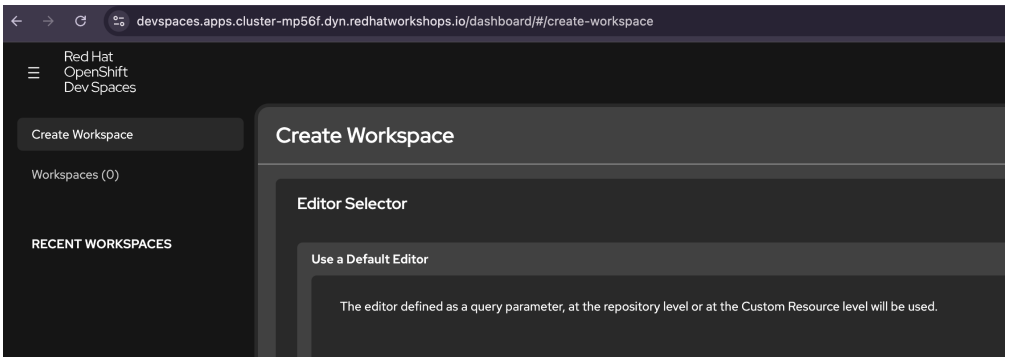
```
oc get checluster devspaces -n openshift-devspaces \
  -o jsonpath='{.status.cheURL}{"\n"}'
```

Open this URL in your browser. You will authenticate using OpenShift OAuth with your OpenShift credentials (e.g., `admin` / `<cluster-admin-password>`).

<Add instruction saying allow the permissions>



<Add instruction saying you should see this page>



You can also access Dev Spaces from the OpenShift Console:

<Optionally> . Click the application launcher icon in the top right of the OpenShift Console. . Select **Red Hat OpenShift Dev Spaces**. . You will be automatically authenticated with your current OpenShift session.

Appendix: Useful Configuration Commands

The following commands are reference commands for administrators. Do not run them during the lab unless an instructor asks you to change the Dev Spaces configuration.

```
# Change storage strategy
oc patch checluster devspaces -n openshift-devspaces \
  --type='merge' \
  -p '{"spec":{"devEnvironments":{"storage":{"pvcStrategy":"per-workspace"}}}}'
```

```
# Change max workspaces per user
oc patch checluster devspaces -n openshift-devspaces \
  --type='merge' \
  -p '{"spec":{"devEnvironments":{"maxNumberOfWorkspacesPerUser":10}}}'

# Read a specific setting
oc get checluster devspaces -n openshift-devspaces \
  -o jsonpath='{.spec.devEnvironments.storage.pvcStrategy}{"\n"}'
```

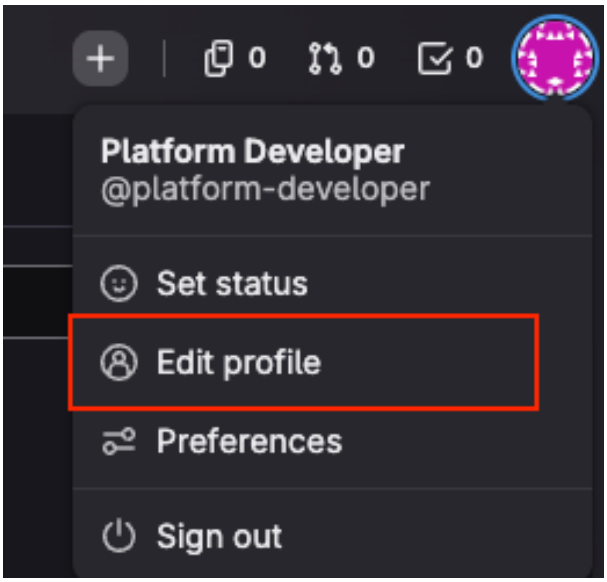
Lab 5

GitLab OAuth for SCM Integration

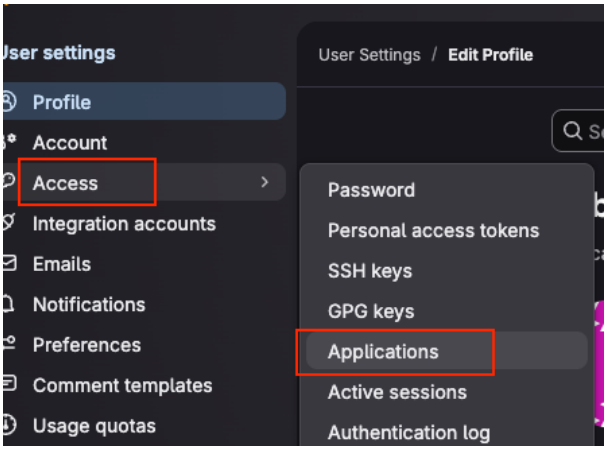
Connecting Dev Spaces to GitLab allows automatic Git credential injection into workspaces — developers can clone, push, and pull without manually configuring tokens.

Create a GitLab OAuth Application

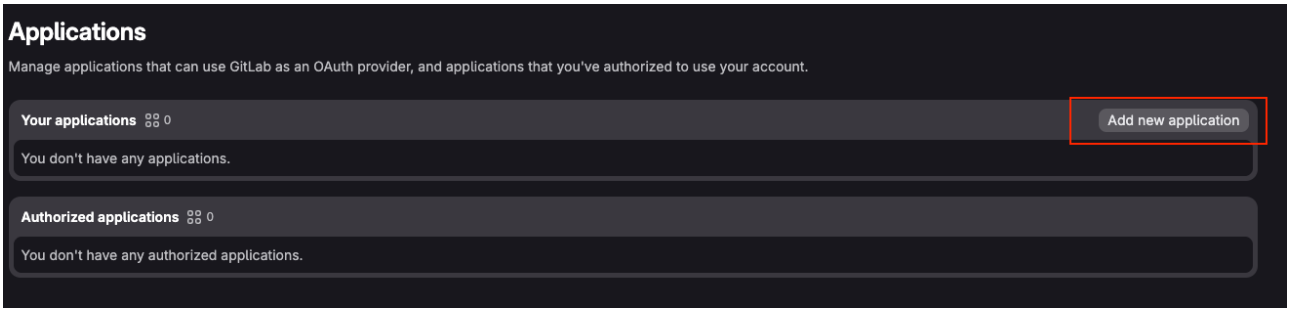
1. Open **GITLAB_URL** from your lab information page and authenticate with your platform-developer/openshift credentials.
2. Click your avatar in the upper-right corner.
3. Select **menu:Edit** profile:



4. Select **menu:Access** > **Applications** to open the User Settings applications page.



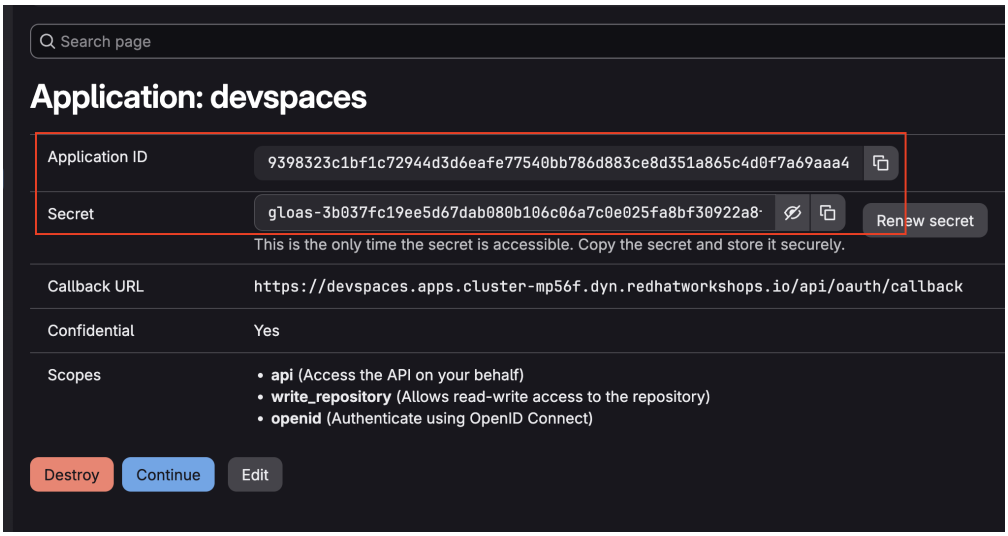
5. Click btn:[Add new application].



6. Create the application with the following settings:

Name	`devspaces`
Redirect URI	`https://devspaces.{openshift_cluster_ingress_domain}/api/oauth/callback`
Confidential	Checked
Scopes	`api`, `write_repository`, `openid`

7. Click btn:[Save application].
8. Note the **Application ID** and **Secret**



Create the OAuth Secret

Create a Kubernetes secret that Dev Spaces will use to authenticate against GitLab. The labels and annotations are required — Dev Spaces discovers the secret automatically based on them.

Replace `GITLAB_APPLICATION_ID` and `GITLAB_APPLICATION_SECRET` with the values from the GitLab OAuth application before running the command.

```
# Get the GitLab URL dynamically from the cluster
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

echo "Using GitLab URL: $GITLAB_URL"

oc create secret generic gitlab-oauth-config \
  -n openshift-devspaces \
  --from-literal=id="$GITLAB_APPLICATION_ID" \
  --from-literal=secret="$GITLAB_APPLICATION_SECRET"

oc annotate secret gitlab-oauth-config \
  -n openshift-devspaces \
  che.eclipse.org/oauth-scm-server=gitlab \
  che.eclipse.org/scm-server-endpoint="$GITLAB_URL"

oc label secret gitlab-oauth-config \
  -n openshift-devspaces \
  app.kubernetes.io/part-of=che.eclipse.org \
  app.kubernetes.io/component=oauth-scm-configuration

echo "GitLab OAuth secret created successfully"
```

Register GitLab in the CheCluster

Add the `gitServices` section to your CheCluster CR:

```
# Get the GitLab URL dynamically from the cluster
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

echo "Registering GitLab URL: $GITLAB_URL"

oc patch checluster devspaces -n openshift-devspaces \
  --type='merge' \
  -p "{\"spec\":{\"gitServices\":{\"gitlab\":[{\"secretName\":\"gitlab-oauth-
config\",\"endpoint\":\"$GITLAB_URL\"}]}}}"
```

When a user opens a workspace that references a GitLab repository, they will be prompted to authenticate with GitLab via OAuth. This grants push/pull access using their personal GitLab

credentials.

Mounting Shared Secrets into Workspaces

Dev Spaces can automatically mount Kubernetes Secrets (and ConfigMaps) into every workspace pod using labels and annotations. This is useful for providing API keys, service credentials, or shared configuration to all developers without them having to configure it manually.

How It Works

Any Secret or ConfigMap in the `openshift-devspaces` namespace with the label `app.kubernetes.io/part-of: che.eclipse.org` and the annotation `controller.devfile.io/mount-as` will be automatically mounted into workspace containers.

Mount options:

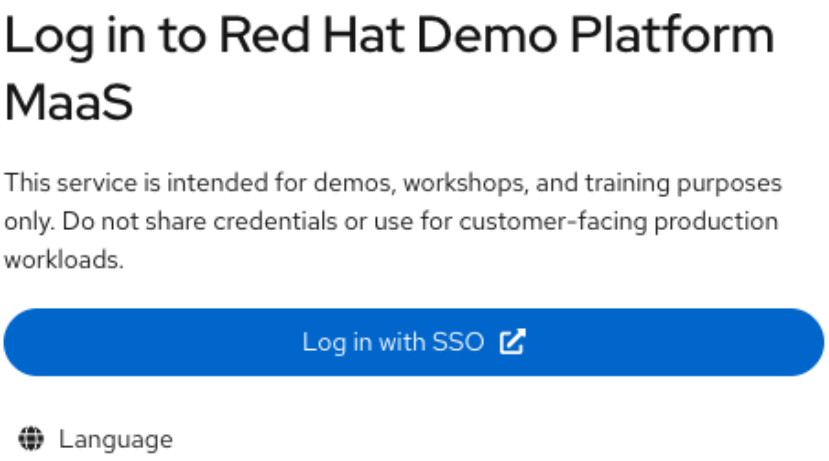
- `mount-as: env` — Inject as environment variables
- `mount-as: file` — Mount as files at a specified path
- `mount-as: subpath` — Mount individual keys as separate files

Example: AI Code Assistant (Continue) Credentials

This example creates a secret that injects LLM API credentials as environment variables into every workspace. These can be provided to an extension such as Roo, Cline, or Continue to enable LLM-assisted coding.

First, obtain an API key from the Red Hat Demo Platform MaaS.

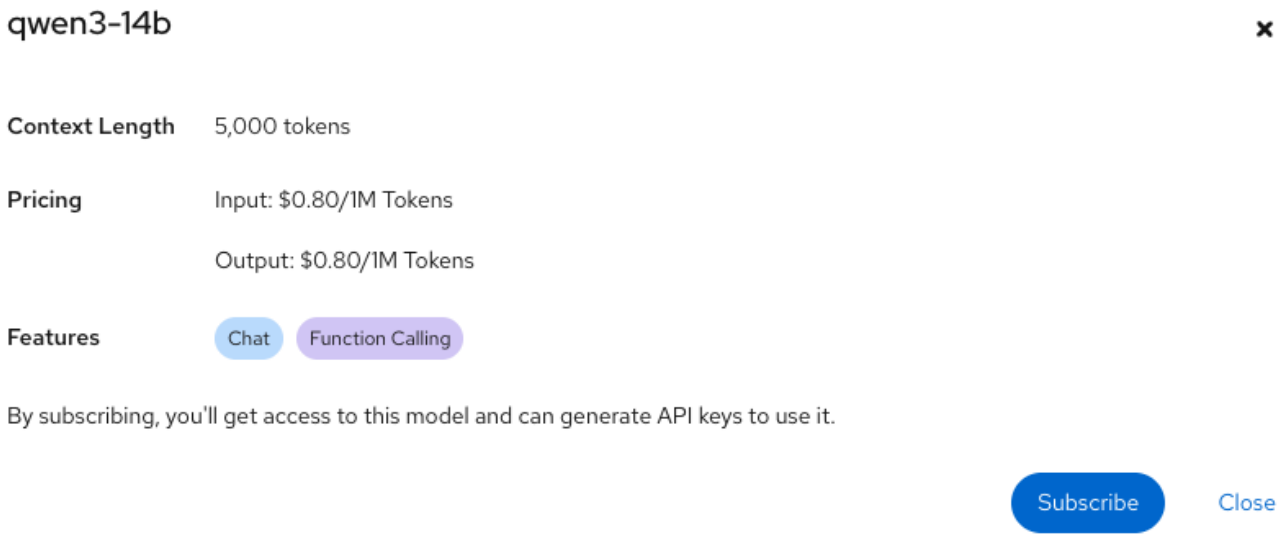
Log in with your SSO credentials. If you do not have access to this service, ask the instructor for the workshop-provided LLM endpoint and API key.



If you don't have an API key, you can create one by following these steps:

1. Navigate to **Models** and find **qwen3-14b**

2. Click **Subscribe** to subscribe to the model



3. If the model already shows an active subscription, continue to the API key step
4. Go to **API Keys** and click **Create API Key**
5. Provide a name for the key and select the **qwen3-14b** subscription
6. Save the generated key somewhere safe

If you already have an API key, you can use it by following these steps:

1. Go to **API Keys** and select the API Key subscribed to the **qwen3-14b** model
2. Click the **show key** button to view the API key
3. Copy the generated key

Then create the secret:

```
oc create secret generic continue-llm-credentials \
  -n openshift-devspaces \
  --from-literal=LLM_API_KEY="REPLACE_API_KEY" \
  --from-literal=LLM_BASE_URL="https://maas-rhdp.apps.maas.redhatworkshops.io/v1"

oc annotate secret continue-llm-credentials \
  -n openshift-devspaces \
  controller.devfile.io/mount-as=env

oc label secret continue-llm-credentials \
  -n openshift-devspaces \
  app.kubernetes.io/part-of=che.eclipse.org \
  app.kubernetes.io/component=workspaces-config

echo "LLM credentials secret created successfully"
```

Replace `REPLACE_API_KEY` with your actual API key from the MaaS platform before

running the command.

Every new workspace will now have `LLM_API_KEY` and `LLM_BASE_URL` available as environment variables. The devfile's `postStart` command can use these to configure the Continue extension automatically (see Devfiles).

Lab 6

Overview

In this lab you will create a Dev Spaces workspace for the ETX App Base application, configure it with a custom devfile, verify Git and AI assistant integration, and run Quarkus in development mode connected to external PostgreSQL and Kafka services deployed in the cluster.

Access Dev Spaces Dashboard

Get the Dev Spaces dashboard URL:

```
oc get checluster devspaces -n openshift-devspaces \
  -o jsonpath='{.status.cheURL}{"\n"}'
```

Open this URL in your browser and log in with your OpenShift credentials.

Dev Spaces on OpenShift uses OpenShift OAuth for authentication by design.

Step 1: Create Initial Workspace

The `etx_app_base_app` repository does not include a devfile yet. You will create a temporary workspace to add the devfile to the repository.

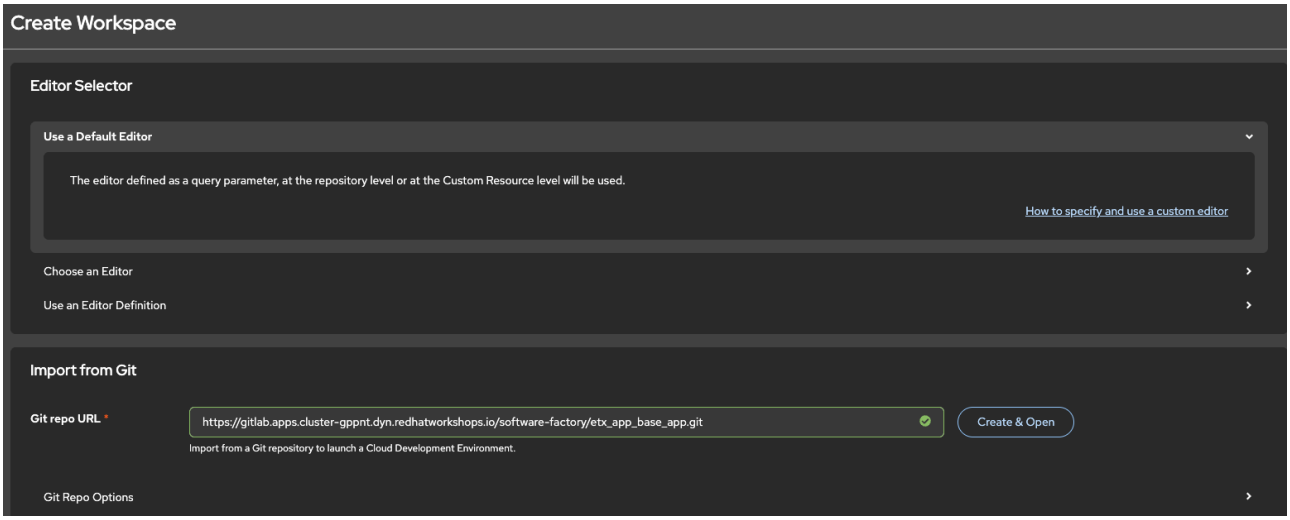
From the Dev Spaces dashboard:

1. Click btn:[Create Workspace]
2. Select **Import from Git**
3. Get the gitlab url from below command:

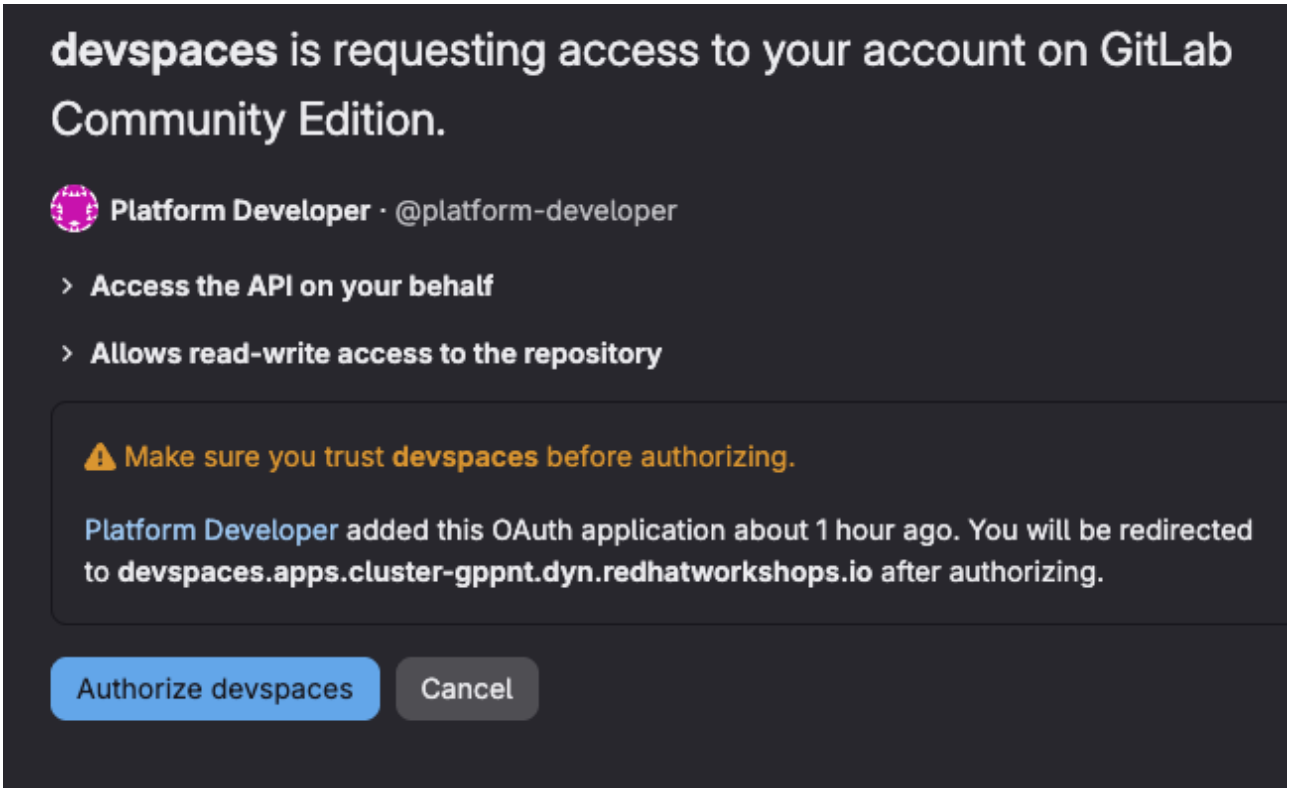
```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo $GITLAB_URL
```

4. Paste the repository URL:

```
{GITLAB_URL}/software-factory/etx_app_base_app.git
```



5. Click btn:[Create & Open]
6. If prompted to authorize with GitLab, follow the OAuth flow and approve access

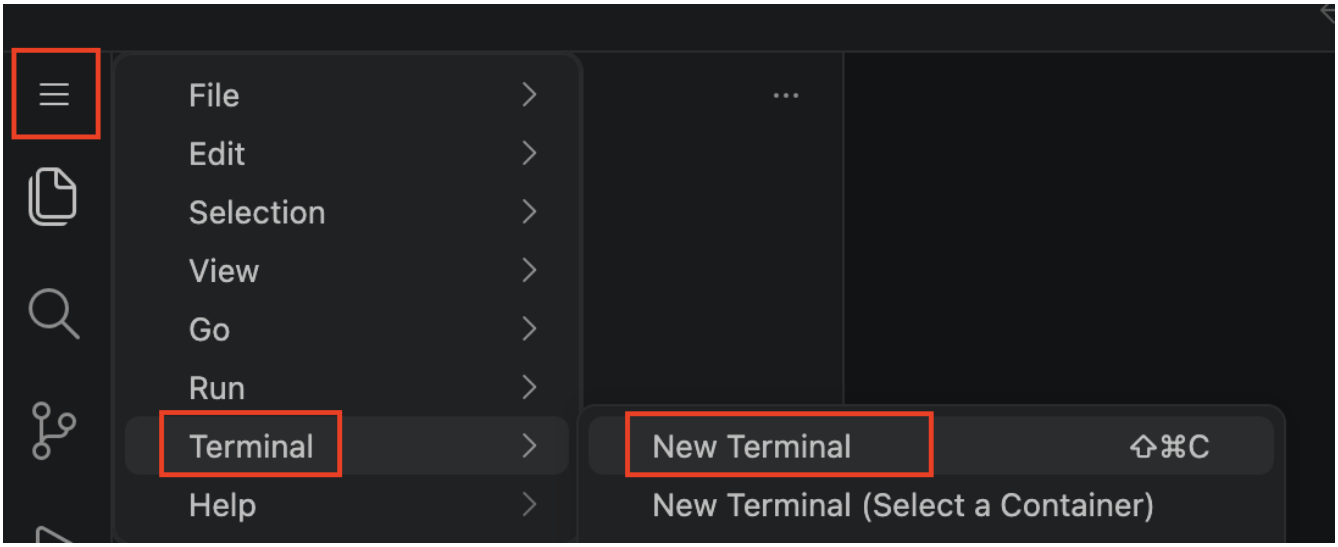


Dev Spaces will create a workspace using the Universal Developer Image (UDI) since there is no devfile in the repository yet.

Step 2: Configure Git

Once the workspace opens, configure Git identity.

Open a terminal from the main menu (hamburger icon in the top-left corner: menu:Terminal[New Terminal]).



Configure Git identity

```
git config --global user.email "platform-developer@example.com"
git config --global user.name "Platform Developer"
```

Step 3: Create the Devfile

Create a `devfile.yaml` file that will configure your development environment.

Create the devfile using below command in the newly opened terminal inside devspaces:

```
cat > devfile.yaml <<'EOF'
schemaVersion: 2.2.0
metadata:
  name: etx-app-base-app
components:
  - name: development-tooling
    container:
      image: quay.io/devfile/universal-developer-image:ubi9-latest
      env:
        - name: QUARKUS_HTTP_HOST
          value: "0.0.0.0"
        - name: MAVEN_OPTS
          value: "-Dmaven.repo.local=/home/user/.m2/repository"
        # External service connections
        - name: POSTGRESQL_HOST
          value: "parasol-db.etx-app-dev.svc.cluster.local"
        - name: POSTGRESQL_DATABASE
          value: "parasol"
        - name: POSTGRESQL_USER
          value: "parasol"
        - name: POSTGRESQL_PASSWORD
          value: "parasol"
        - name: KAFKA_BOOTSTRAP_SERVERS
          value: "parasol-kafka-kafka-bootstrap.etx-app-dev.svc.cluster.local:9092"
```

```
      memoryLimit: 5Gi
      cpuLimit: 2500m
      volumeMounts:
        - name: m2
          path: /home/user/.m2
      endpoints:
        - name: quarkus-dev
          targetPort: 8080
          exposure: public
          protocol: https
        - name: debug
          targetPort: 5005
          exposure: none
  - name: m2
    volume:
      size: 1G
commands:
  - id: package
    exec:
      label: "1. Package the application"
      component: development-tooling
      commandLine: "./mvnw package"
      group:
        kind: build
        isDefault: true
  - id: start-dev
    exec:
      label: "2. Start Development mode (Hot reload + debug)"
      component: development-tooling
      commandLine: "./mvnw compile quarkus:dev"
      group:
        kind: run
        isDefault: true
  - id: init-continue
    exec:
      label: "Initialize Continue config"
      component: development-tooling
      workingDir: /home/user
      commandLine: |
        mkdir -p /home/user/.continue
        cat > /home/user/.continue/config.yaml << EOF
        name: Continue Config
        version: 0.0.1
        models:
          - name: qwen3-14b
            provider: vllm
            model: qwen3-14b
            apiKey: ${LLM_API_KEY}
            apiBase: ${LLM_BASE_URL}
            roles:
              - chat
```

```
        EOF
      group:
        kind: build
events:
  postStart:
    - init-continue
EOF
```

Verify the devfile was created:

```
ls -la devfile.yaml
cat devfile.yaml | head -20
```

Step 4: Commit and Push the Devfile

Create a feature branch, commit the devfile, and push to GitLab:

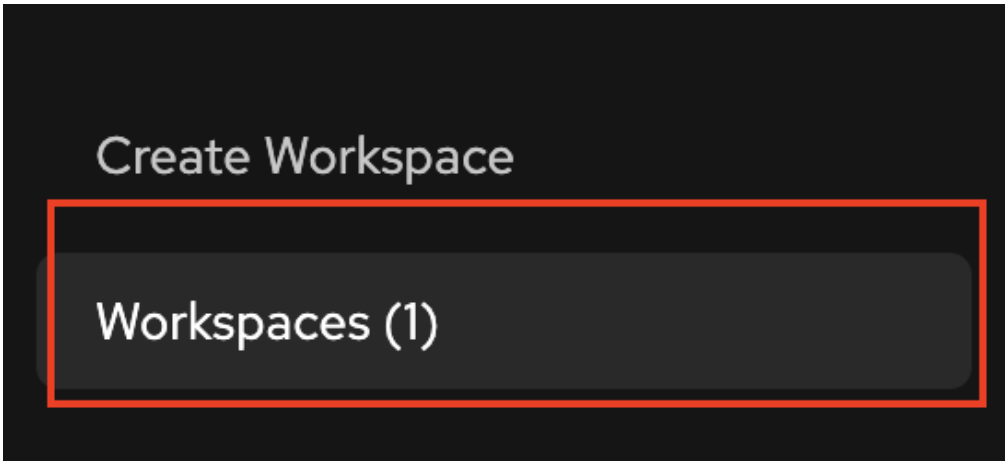
```
# Create and switch to feature branch
git checkout -b feature/add-devfile

# Add and commit the devfile
git add devfile.yaml
git commit -m "feat: add devfile for Dev Spaces configuration"

# Push to GitLab
git push origin feature/add-devfile
```

Step 5: Recreate Workspace from Devfile

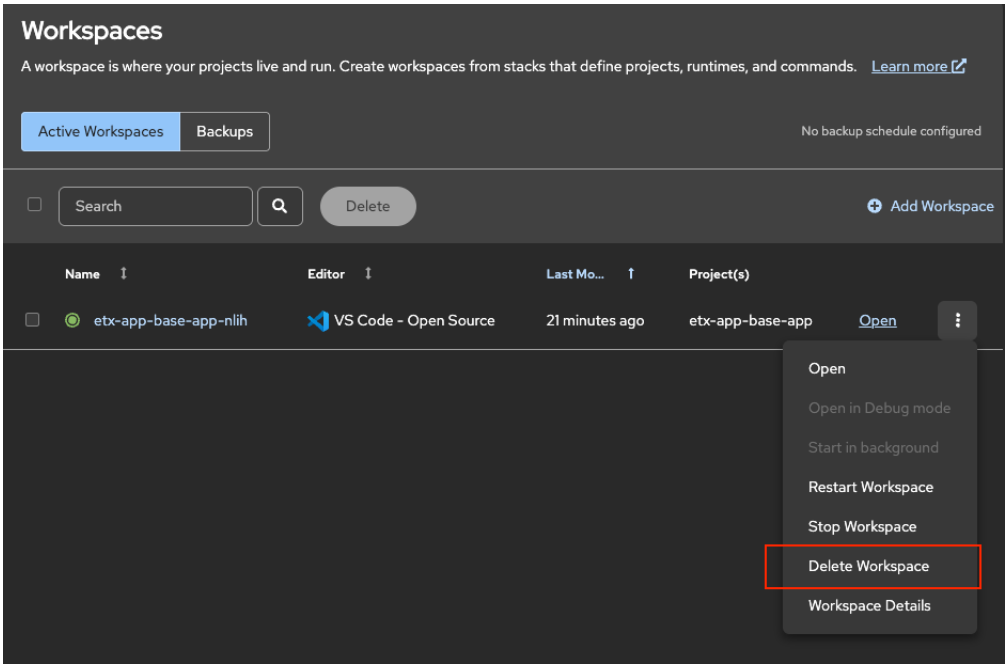
Now delete the temporary workspace and create a new one that will use your devfile.

Delete the Temporary Workspace

1. Return to the Dev Spaces dashboard browser tab
2. Find your workspace in the list



3. Click the three dots (⋮) menu → **Delete Workspace**



4. Confirm deletion and wait until the workspace disappears

Create Workspace from Devfile

You can create a workspace using either a direct URL (faster) or the UI (manual).

Direct URL (Recommended)

Use a direct URL to create a workspace from a specific branch.

Get the required URLs:

```
# Get Dev Spaces URL
DEVSPACES_URL="https://$(oc get checluster devspaces -n openshift-devspaces -o
jsonpath='{.status.cheURL}' | sed 's|https://||')"

# Get GitLab URL
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

# Construct the direct workspace URL
echo "${DEVSPACES_URL}#${GITLAB_URL}/software-factory/etx_app_base_app/-
/tree/feature/add-devfile"
```

Open the generated URL in your browser. Dev Spaces will:

1. Automatically import the repository from the feature/add-devfile branch
2. Detect and use the devfile.yaml
3. Configure the workspace with all environment variables and settings

This skips the manual "Create Workspace → Import from Git" flow.

Manual UI

Create a workspace through the Dev Spaces dashboard.

1. Click btn:[Create Workspace]
2. Select **Import from Git**
3. Paste the repository URL (use the actual GitLab URL from your cluster):

```
{gitlab_url}/software-factory/etx_app_base_app/-/tree/feature/add-devfile
```

4. Click btn:[Create & Open]

Dev Spaces will detect the devfile in the repository and use it to configure your workspace with:

- PostgreSQL and Kafka connection variables
- Maven cache persistence
- Quarkus development endpoints
- Continue AI assistant initialization

Wait for the workspace to fully start. This may take 2-3 minutes as the container image is pulled and configured.

The direct URL method is faster and can be bookmarked for quick access. You can also specify different branches by changing `/tree/feature/add-devfile` to `/tree/<branch-name>`.

Verify Workspace Configuration

Once the workspace opens, verify it was configured correctly with your devfile.

Dev Spaces automatically clones the repository when you create a workspace from a Git URL. The repository will be available in `/projects/etx_app_base_app`.

Verify Repository Clone

Open a terminal and verify the repository was cloned correctly.

Or, you can also verify from a workspace terminal, opening from the burger menu on the top right of the IDE, then selecting menu:Terminal[New Terminal]:

The terminal opens in `/projects/etx_app_base_app` by default. If you've navigated elsewhere, use `cd /projects/etx_app_base_app` to return.

```
# Verify current branch and recent commits
git branch --show-current
git log --oneline -3
```

```
ls -la devfile.yaml
```

You should see `feature/add-devfile` as the current branch and the `devfile.yaml` file in the repository root.

Check Environment Variables

Verify the environment variables from your devfile are present:

```
echo "POSTGRESQL_HOST: $POSTGRESQL_HOST"
echo "POSTGRESQL_DATABASE: $POSTGRESQL_DATABASE"
echo "POSTGRESQL_USER: $POSTGRESQL_USER"
echo "KAFKA_BOOTSTRAP_SERVERS: $KAFKA_BOOTSTRAP_SERVERS"
```

You should see:

```
POSTGRESQL_HOST: parasol-db.etx-app-dev.svc.cluster.local
POSTGRESQL_DATABASE: parasol
POSTGRESQL_USER: parasol
KAFKA_BOOTSTRAP_SERVERS: parasol-kafka-kafka-bootstrap.etx-app-
dev.svc.cluster.local:9092
```

If these variables are **empty**, the workspace was not created with your devfile. This can happen if:

- The devfile was not pushed to the feature/add-devfile branch in Step 4
- The workspace was created before the devfile was pushed

Go back to Step 5 and recreate the workspace after ensuring the devfile is in the repository.

Test Service Connectivity

Verify network connectivity to PostgreSQL:

```
timeout 3 bash -c "echo > /dev/tcp/$POSTGRESQL_HOST/5432" && echo "PostgreSQL
connection successful" || echo "PostgreSQL connection failed"
```

Expected: `PostgreSQL connection successful`

If you get connection errors, verify you completed Lab 2 - Deploy Supporting Services.

Run Quarkus in Development Mode

When you open a terminal in the workspace, you are automatically positioned in the project directory.

Verify your current location and start Quarkus development mode:

```
# Verify you're in the project directory
pwd

# Start Quarkus in development mode
./mvnw compile quarkus:dev
```

Expected output from `pwd`:

```
/projects/etx_app_base_app
```

Quarkus connects to the external PostgreSQL and Kafka services using the environment variables from your devfile.

Wait for the application to start. You should see output similar to:

```
Listening for transport dt_socket at address: 5005
__  ____  __  _____   ___  __ ____  ______
 --/ __ \/ / / / _ | / _ \/ //_/ / / / __/
 -/ /_/ / /_/ / __ |/ , _/ ,< / /_/ /\ \
--\___\_\____/_/ |_/_/|_/_/|_|\____/___/

INFO  [io.sma.rea.mes.kafka] SRMSG18229: Configured topics for channel 'emails-in':
[intake]
INFO  [io.sma.rea.mes.kafka] SRMSG18258: Kafka producer kafka-producer-emails-out,
connected to Kafka brokers 'parasol-kafka-kafka-bootstrap.etx-app-
dev.svc.cluster.local:9092'
INFO  [io.sma.rea.mes.kafka] SRMSG18257: Kafka consumer kafka-consumer-emails-in,
connected to Kafka brokers 'parasol-kafka-kafka-bootstrap.etx-app-
dev.svc.cluster.local:9092'
INFO  [io.quarkus] parasol-reimagined 1.0.0-SNAPSHOT on JVM (powered by Quarkus
3.17.5) started in 7.298s. Listening on: http://0.0.0.0:8080
INFO  [io.quarkus] Profile dev activated. Live Coding activated.

--
Tests paused
Press [e] to edit command line args, [r] to resume testing, [h] for more options>
```

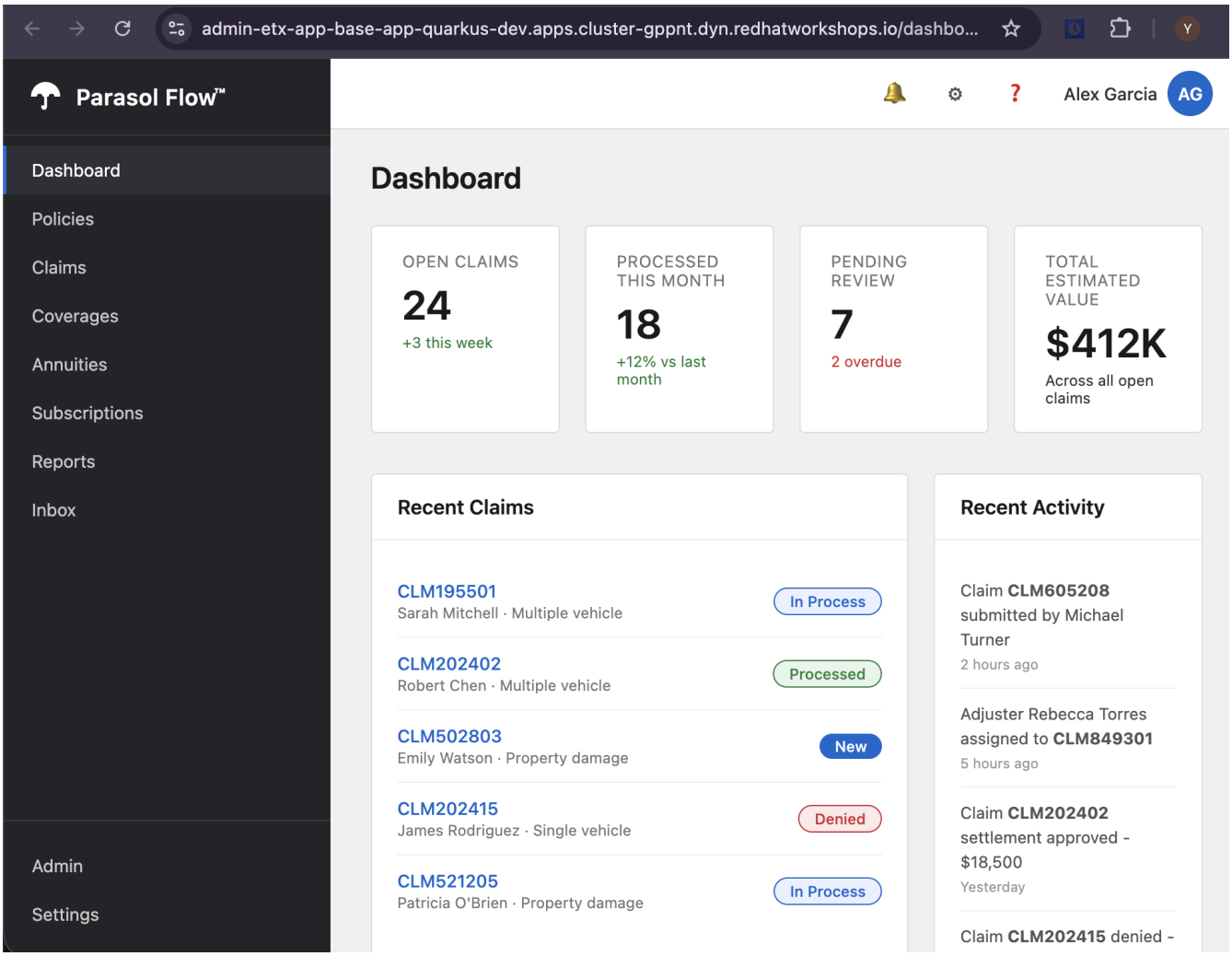
Key indicators of successful startup:

- **Debug port listening**: `Listening for transport dt_socket at address: 5005`
- **Kafka connected**: Messages showing connection to `parasol-kafka-kafka-bootstrap.etx-app-dev.svc.cluster.local:9092`
- **Application started**: `Listening on: http://0.0.0.0:8080`
- **Live Coding activated**: Hot reload is enabled

Access the Application

When Quarkus starts, a **popup notification** appears in the bottom-right corner with a button to open the application endpoint.

Click **Open in New Tab** or **Open in Preview** to access the running application.



If you miss the popup, press `Ctrl+C` to stop Quarkus, then run `./mvnw compile quarkus:dev` again to restart and see the notification.

The endpoint URL follows this structure:

```
https://<username>-<project-name>-quarkus-dev.apps.<cluster-domain>/
```

For example:

```
https://admin-etx-app-base-app-quarkus-dev.apps.cluster-
grsml.dynamic2.redhatworkshops.io/
```

You can also find the URL in the **Endpoints** panel (usually in the bottom-left sidebar).

External Services Configuration

Your application connects to:

- **PostgreSQL**: `parasol-db.etx-app-dev.svc.cluster.local:5432`
  - Database: `parasol`
  - Credentials: `parasol/parasol`
- **Kafka**: `parasol-kafka-kafka-bootstrap.etx-app-dev.svc.cluster.local:9092`
  - Topic: `intake`

These are shared services across all development workspaces, deployed in Deploy Supporting Services.

Workspace Lifecycle

Useful operations from the Dev Spaces dashboard:

Stop workspace (preserves files on PVC):

- Click three dots (⋮) → **Stop Workspace**
- Pod is deleted, PVC is retained

Start workspace:

- Click three dots (⋮) → **Start in background**
- Your files, Maven cache, and Git state persist

Delete workspace:

- Click three dots (⋮) → **Delete Workspace**
- Removes pod and PVC